

ONLINE BOOKSTORE

An E-Commerce Platform for Buying and Selling Books Online

Phase 2: Design Software Design Document

Members	Anagha N (PES1UG24CS058) Manasvi B (PES1UG24CS261) Meda Sahasra Siri (PES1UG24CS267) Monika (PES1UG24CS274)
Course	Software Engineering
Technology Stack	MERN (MongoDB, Express.js, React, Node.js)
Repository	https://github.com/Ana-9211/Software-Engineering-Project-
Based on	Phase 1 Requirements (unchanged)
Document Version	1.0 (Phase 2 draft for team review)

Contents

1. Introduction	3
1.1 Purpose and scope	3
1.2 Source of truth and traceability	3
1.3 Conventions	3
2. Design Principles	4
3. Project Structure and Conventions	5
4. Detailed Module Design	6
4.1 Backend modules	6
4.2 Frontend modules	10
4.3 Key algorithms and rules	11
4.4 Search Design	14
5. UML Models	16
5.1 Domain class diagrams	16
5.2 Service-layer class diagram	22
5.3 Component diagrams	23
5.4 Deployment diagram	26
5.5 Sequence diagrams	28
5.6 Activity diagrams	41
6. API Definition	44
6.1 Conventions	44
6.2 Error code catalogue	44
6.3 Endpoint summary	46
6.4 Endpoint definitions	50
6.5 OpenAPI specification (VFR-15)	70
6.6 Role-based access matrix (FR-29)	71
7. ABI and Internal Interface Definition	74
7.1 What "ABI" means in this project	74
7.2 Contracts	74
7.3 Who checks the contracts	74
8. Database Design	76
8.1 ER diagrams	76
8.2 Data dictionary	80
8.3 How the design supports the requirements	84
8.4 Transactions	84
8.5 Initial data	85
8.6 Image Storage with GridFS	85
9. Division of Work	87
9.1 Primary ownership	87
9.2 Phase 2 document responsibilities	87
9.3 Review and support responsibilities	88
9.4 Cross-cutting Phase 3 tasks	88
10. Traceability: Requirements to Modules and API	89
11. Assumptions and Open Items	91

1. Introduction

1.1 Purpose and scope

This Software Design Document describes how the Online Bookstore will be built. It refines the High-Level Architecture (`../architecture/High-Level-Architecture.md`) into module design, UML models, the REST API, the internal interface contract (the project's "ABI"), the database design and the division of work. It is detailed enough for the team to start Phase 3.

1.2 Source of truth and traceability

Phase 1 (`docs/phase-1/`) is unchanged and is the source of truth. This document uses the Phase 1 identifiers exactly (FR-01 to FR-30, VFR-01 to VFR-15, UC-01 to UC-25, and the six actors). The traceability chain is:

Requirement (FR or VFR) → Use case (UC) → Architecture component (AC) → Module (BM or FM) → Endpoint (API) → Database collection → Test case (TC)

Identifiers defined in the Architecture document (AC, BM, FM, I, S, D, A) are used here without change. Where Phase 1 is ambiguous this document follows the design decisions and assumptions D-01 to D-15 and ambiguities A-01 to A-14 listed in the Architecture document, Section 11.

1.3 Conventions

Item	Convention
Money	Integer in minor currency units; two decimals only when displayed
Dates and times	ISO 8601 in UTC
Identifiers	MongoDB ObjectId as a 24-character hex string
Priorities and methods	As in Phase 1 (M, S, C; T, I, A, D)
Naming	Collections in snake_case plural; JSON fields in camelCase; endpoints in kebab-case

2. Design Principles

Principle	How it is applied in this project
Modularity	Sixteen backend modules, each owning its data and exposing a small set of service functions (Section 4.1). Frontend is split into eleven modules by use-case group
Separation of concerns	Routes and controllers handle HTTP only; services hold business rules; repositories hold database access; adapters hold external services
Low coupling	Modules call each other only through service functions listed as interfaces (I-04 to I-24). No module reads another module's collection directly. Payment and email providers are behind interfaces
High cohesion	Everything about one entity lives in one module. Stock logic is only in Inventory, email only in Notification, payment calls only in Payment
Reusable services and components	Shared pieces: <code>AppError</code> , <code>validate(schema)</code> , <code>requireRole</code> , <code>computeSummary</code> , <code>OrderStatusPolicy</code> , and on the frontend <code>ApiClient</code> , <code>Pagination</code> , <code>StatusBadge</code> and form components
Secure by design	Authentication and role checks are middleware applied per route, ownership is checked in services, secrets come from the environment, card data never enters the system (Architecture Section 8)
Testability	Services receive repositories and adapters as constructor arguments, so tests use fakes (<code>FakeGateway</code> , <code>ConsoleMailSender</code> , in-memory or test-database repositories); the clock is injectable for lockout and expiry tests

3. Project Structure and Conventions

This is the proposed layout for Phase 3. It is a design proposal, not existing code.

```
bookstore/
  server/
    src/
      app.js           # builds the Express app and wires modules
      config/          # reads and checks environment variables
      middleware/       # authGuard, requireRole, validate, csrfProtection,
                        # sanitize, ratelimiter, errorHandler, requestLogger
      modules/
        auth/ user/ catalog/ book-management/ cart/ wishlist/ order/ payment/
        review/ seller/ inventory/ admin/ reporting/ notification/ media/
        <module>.routes.js <module>.controller.js <module>.service.js
        <module>.repository.js <module>.schemas.js
      adapters/
        payment/       # PaymentGateway interface, provider adapter, fakeGateway
        mail/          # MailSender interface, provider adapter, consoleMailSender
      models/          # Mongoose schemas
      utils/           # AppError, money helpers, redact
      scripts/         # seed-admin, seed-demo-data
      tests/           # unit, integration, api
  client/
    src/
      api/             # ApiClient (credentials, CSRF header, error mapping)
      auth/            # AuthContext, ProtectedRoute, RoleRoute
      components/      # shared components
      modules/         # auth, catalog, cart, wishlist, checkout, orders,
                        # reviews, profile, seller, admin
    routes.jsx
  docs/
```

Convention	Rule
Error handling	Services throw <code>AppError(code, status, message, details)</code> . Controllers do not catch; the <code>errorHandler</code> middleware converts every error to the uniform error object. Unknown errors become 500 <code>INTERNAL_ERROR</code> without internal details
Validation	Each route declares schemas for body, query and params. Validation runs before the controller. Unknown fields are rejected
Transactions	A service that changes more than one document starts a MongoDB session and passes it to repositories (I-03)
Logging	Through one logger with the redaction function; no <code>console.log</code> in modules
Configuration	Only <code>config/</code> reads environment variables
Lint and tests	ESLint with zero errors and Jest coverage reporting in CI (VFR-13)

4. Detailed Module Design

4.1 Backend modules

Each table lists the classes, the key functions with their parameters, the error codes the module can raise, and the endpoints it serves. The responsibilities and dependencies are in the Architecture document, Section 4.2.

BM-01 Authentication

Item	Design
Classes	AuthController, AuthService, TokenService, PasswordService, AuthTokenRepository
Key functions	register(dto) verifyEmail(token) resendVerification(email) login(email, password) logout(userId) getCurrentUser(userId) forgotPassword(email) resetPassword(token, newPassword) invalidateSessions(userId)
Error codes	VALIDATION_ERROR, EMAIL_TAKEN, TOKEN_INVALID_OR_EXPIRED, INVALID_CREDENTIALS, EMAIL_NOT_VERIFIED, ACCOUNT_SUSPENDED, ACCOUNT_LOCKED
Endpoints	API-02, API-03, API-04, API-05, API-06, API-07, API-08, API-09
Requirements	FR-01, FR-02, FR-03, FR-24, FR-29, VFR-04, VFR-07

BM-02 User and Profile

Item	Design
Classes	ProfileController, UserService, UserRepository
Key functions	getProfile(userId) updateProfile(userId, dto) addAddress(userId, dto) updateAddress(userId, addressId, dto) removeAddress(userId, addressId) findByEmail(email) listUsers(filter, page) setStatus(userId, status) grantRole(userId, role, sellerProfile)
Error codes	VALIDATION_ERROR, NOT_FOUND, ADDRESS_LIMIT
Endpoints	API-10, API-11, API-12, API-13, API-14
Requirements	FR-18, FR-24, FR-19

BM-03 Catalog

Item	Design
Classes	CatalogController, CatalogService, CategoryService, BookRepository, CategoryRepository
Key functions	listBooks(query) getBook(bookId) listCategories() buildSearchFields(title, author, isbn) recalculateRating(bookId, session) createCategory(name) renameCategory(id, name) deleteCategory(id)
Error codes	VALIDATION_ERROR, NOT_FOUND, CATEGORY_EXISTS, CATEGORY_IN_USE
Endpoints	API-15, API-16, API-18, API-55, API-56, API-57
Requirements	FR-04, FR-05, FR-06, FR-07, FR-26, VFR-01, VFR-02

BM-04 Cart

Item	Design
Classes	CartController, CartService, CartRepository
Key functions	getCart(userId) addItem(userId, bookId, qty) updateItem(userId, bookId, qty) removeItem(userId, bookId) computeSummary(items) clear(userId, session)
Error codes	VALIDATION_ERROR, BOOK_NOT_FOUND, ITEM_NOT_IN_CART, INSUFFICIENT_STOCK
Endpoints	API-20, API-21, API-22, API-23
Requirements	FR-08, FR-09, FR-12

BM-05 Wishlist

Item	Design
Classes	WishlistController, WishlistService, WishlistRepository
Key functions	getWishlist(userId) addBook(userId, bookId) removeBook(userId, bookId)
Error codes	BOOK_NOT_FOUND
Endpoints	API-24, API-25, API-26
Requirements	FR-10

BM-06 Order

Item	Design
Classes	OrderController, OrderService, OrderRepository, OrderStatusPolicy, ReservationSweeper
Key functions	createOrder(userId, addressInput) confirmPayment(userId, orderId, paymentRef) handleGatewayEvent(rawBody, signature) listOrders(userId, page) getOrder(userId, orderId) cancelOrder(userId, orderId) releaseExpiredReservations() deriveOrderStatus(items) applyItemStatus(orderId, itemId, sellerId, status)
Error codes	VALIDATION_ERROR, NOT_FOUND, INSUFFICIENT_STOCK, PAYMENT_VERIFICATION_FAILED, ORDER_FAILED_REFUNDED, ORDER_NOT_CANCELLABLE, PAYMENT_GATEWAY_ERROR
Endpoints	API-27, API-28, API-30, API-31, API-32
Requirements	FR-11, FR-12, FR-13, FR-14, FR-15, FR-16, FR-30, VFR-14

BM-07 Payment

Item	Design
Classes	PaymentController (webhook only), PaymentService, PaymentRepository, PaymentGateway (interface)
Key functions	createPaymentSession(order) verifyPayment(order, paymentRef) verifyWebhook(rawBody, signature) refund(payment, amount)
Error codes	PAYMENT_GATEWAY_ERROR, PAYMENT_VERIFICATION_FAILED, INVALID_SIGNATURE
Endpoints	API-29
Requirements	FR-13, FR-16, VFR-08, VFR-14

BM-08 Review

Item	Design
Classes	ReviewController, ReviewService, ReviewRepository

Item	Design
Key functions	createReview(userId, bookId, dto) listReviews(bookId, page) listAllReviews(filter, page) deleteReview(reviewId)
Error codes	VALIDATION_ERROR, NOT_ELIGIBLE, ALREADY_REVIEWED, NOT_FOUND
Endpoints	API-17, API-33, API-58, API-59
Requirements	FR-17, FR-27

BM-09 Seller

Item	Design
Classes	SellerController, SellerService, FulfilmentService, SellerApplicationRepository
Key functions	apply(userId, dto) getMyApplication(userId) decideApplication(adminId, id, decision, reason) listSellerOrders(sellerId, filter, page) updateItemStatus(sellerId, orderId, itemId, status)
Error codes	VALIDATION_ERROR, NOT_FOUND, APPLICATION_EXISTS, ALREADY_SELLER, NOT_PENDING, INVALID_STATUS_TRANSITION
Endpoints	API-34, API-35, API-42, API-43, API-50, API-51
Requirements	FR-19, FR-22, FR-25

BM-10 Inventory

Item	Design
Classes	InventoryService (no controller; called by other services), BookRepository
Key functions	checkAvailability(items) reserve(items, session) finalizeReservation(items, session) releaseReservation(items, session) restore(items, session) setStock(sellerId, bookId, stock, threshold) evaluateLowStock(book)
Error codes	INSUFFICIENT_STOCK, NOT_FOUND
Endpoints	API-40
Requirements	FR-09, FR-16, FR-21, FR-30, VFR-14

BM-11 Admin

Item	Design
Classes	AdminController, AdminService
Key functions	listUsers(filter, page) setUserStatus(adminId, userId, status) listApplications(status, page) listPendingBooks(status, page) manageCategory(action, dto) moderateReview(reviewId)
Error codes	NOT_FOUND, NOT_PENDING, CANNOT_MODIFY_SELF, CATEGORY_EXISTS, CATEGORY_IN_USE
Endpoints	API-47, API-48, API-49, API-52
Requirements	FR-19, FR-24, FR-25, FR-26, FR-27, FR-28

BM-12 Reporting

Item	Design
Classes	ReportController, ReportingService, OrderRepository (aggregations)
Key functions	salesReport(sellerId, from, to) platformReport(from, to)
Error codes	INVALID_DATE_RANGE

Item	Design
Endpoints	API-44, API-60
Requirements	FR-23, FR-28

BM-13 Notification

Item	Design
Classes	NotificationController (in-app list), NotificationService, NotificationRepository, MailSender (interface), EmailTemplates
Key functions	<pre> sendVerificationEmail(user, token) sendPasswordResetEmail(user, token) sendOrderConfirmation(order, user) notifyLowStock(book) listForUser(userId, filter, page) markRead(userId, notificationId) retryFailed() </pre>
Error codes	NOT_FOUND
Endpoints	API-45, API-46
Requirements	FR-01, FR-03, FR-14, FR-21

BM-14 Media

Item	Design
Classes	MediaController, MediaService (GridFS)
Key functions	<pre> saveImage(sellerId, file) streamImage(imageId) </pre>
Error codes	INVALID_FILE, FILE_TOO_LARGE, NOT_FOUND
Endpoints	API-19, API-41
Requirements	FR-07, FR-20

BM-15 Platform and Security

Item	Design
Classes	config, helmetConfig, csrfProtection, authGuard, requireRole, validate(schema), sanitize, rateLimiter, errorHandler, requestLogger, HealthController, AppError
Key functions	<pre> authGuard(req) requireRole(...roles) validate(schema) csrfProtection(req) rateLimiter(options) errorHandler(err) redact(logEntry) health() </pre>
Error codes	UNAUTHENTICATED, FORBIDDEN, CSRF_INVALID, VALIDATION_ERROR, RATE_LIMITED, INTERNAL_ERROR
Endpoints	API-01, API-61
Requirements	FR-29, VFR-05, VFR-06, VFR-07, VFR-08, VFR-09, VFR-13, VFR-15

BM-16 Book Management

Item	Design
Classes	BookManagementController, BookManagementService, BookRepository
Key functions	<pre> createListing(sellerId, dto) updateListing(sellerId, bookId, dto) removeListing(sellerId, bookId) listOwnListings(sellerId, status, page) listPendingListings(status, page) decideListing(adminId, bookId, decision, reason) </pre>
Error codes	VALIDATION_ERROR, NOT_FOUND, ISBN_EXISTS_FOR_SELLER, NOT_PENDING
Endpoints	API-36, API-37, API-38, API-39, API-53, API-54
Requirements	FR-20, FR-25

4.2 Frontend modules

FM-01 Authentication UI

Item	Design
Components	RegisterPage, VerifyEmailPage, LoginPage, ForgotPasswordPage, ResetPasswordPage
Screens	S-03, S-04, S-05, S-06, S-07
API calls	API-01, API-02, API-03, API-04, API-05, API-06, API-07, API-08, API-09
Requirements	FR-01, FR-02, FR-03

FM-02 Catalog UI

Item	Design
Components	CatalogPage, SearchBar, FilterPanel, SortSelect, BookCard, Pagination, BookDetailPage, AddToCartButton, WishlistButton
Screens	S-01, S-02
API calls	API-15, API-16, API-17, API-18, API-19, API-21, API-25
Requirements	FR-04, FR-05, FR-06, FR-07

FM-03 Cart UI

Item	Design
Components	CartPage, CartItemRow, OrderSummaryBox
Screens	S-08
API calls	API-20, API-21, API-22, API-23
Requirements	FR-08, FR-09, FR-12

FM-04 Wishlist UI

Item	Design
Components	WishlistPage, WishlistButton, MoveToCartButton
Screens	S-09
API calls	API-21, API-24, API-25, API-26
Requirements	FR-10

FM-05 Checkout and Payment UI

Item	Design
Components	CheckoutPage, AddressSelector, AddressForm, PaymentPage, OrderConfirmationPage
Screens	S-10, S-11, S-12
API calls	API-10, API-12, API-20, API-27, API-28, API-31
Requirements	FR-11, FR-12, FR-13, FR-14

FM-06 Orders UI

Item	Design
Components	OrderListPage, OrderDetailPage, CancelOrderDialog, StatusBadge
Screens	S-13, S-14
API calls	API-30, API-31, API-32
Requirements	FR-15, FR-16

FM-07 Reviews UI

Item	Design
Components	ReviewForm, ReviewList, StarRating
Screens	S-15
API calls	API-17, API-33
Requirements	FR-17

FM-08 Profile and Seller Application UI

Item	Design
Components	ProfilePage, AddressBook, SellerApplicationPage
Screens	S-16, S-17
API calls	API-10, API-11, API-12, API-13, API-14, API-34, API-35
Requirements	FR-18, FR-19

FM-09 Seller Dashboard UI

Item	Design
Components	SellerHome, ListingsPage, ListingForm, ImageUploader, InventoryPage, SellerOrdersPage, SalesReportPage, AlertList
Screens	S-18, S-19, S-20, S-21, S-22, S-23
API calls	API-36, API-37, API-38, API-39, API-40, API-41, API-42, API-43, API-44, API-45, API-46, API-18, API-19
Requirements	FR-20, FR-21, FR-22, FR-23

FM-10 Administrator Dashboard UI

Item	Design
Components	AdminHome, UsersPage, ApprovalsPage, ReviewModerationPage, CategoriesPage, PlatformReportPage
Screens	S-24, S-25, S-26, S-27, S-28, S-29
API calls	API-47, API-48, API-49, API-50, API-51, API-52, API-53, API-54, API-55, API-56, API-57, API-58, API-59, API-60, API-18
Requirements	FR-24, FR-25, FR-26, FR-27, FR-28

FM-11 Shared UI Infrastructure

Item	Design
Components	AppShell, NavBar (role-aware), ProtectedRoute, RoleRoute, ApiClient, AuthContext, ErrorBoundary, Toast, form components
Screens	S-30
API calls	API-01, API-07
Requirements	FR-29

State handling on the frontend: server data is fetched through `ApiClient` and held in component state or a small data-fetching hook; only the authenticated user (`AuthContext`) and a toast message queue are global. The cart count in the navigation bar comes from `GET /api/cart`.

4.3 Key algorithms and rules

Order status derivation (D-07). Item statuses are ordered `Placed < Packed < Shipped < Delivered`. The order status is the lowest status among its non-cancelled items. If every item is `Cancelled` the order is `Cancelled`. Before confirmation the order is `PendingPayment`; if payment fails, is abandoned, the reservation expires or the stock is no longer available, it is `PaymentFailed` (with a failure reason).

```
deriveOrderStatus(items):  
  active = items where fulfilmentStatus != Cancelled  
  if active is empty: return Cancelled  
  return min(active.fulfilmentStatus) using rank Placed < Packed < Shipped < Delivered
```

Status transitions for a seller update. An item may move only to the next step: `Placed` to `Packed`, `Packed` to `Shipped`, `Shipped` to `Delivered`. Anything else returns `409 INVALID_STATUS_TRANSITION`.

Cancellation rule (FR-16). `canCancel(order)` is true when the order is `PendingPayment`, `Placed` or `Packed` and no item is `Shipped` or `Delivered`. An unpaid order is closed by releasing its reservation (no refund). A `Placed` or `Packed` order is cancelled by restoring stock and requesting a refund.

Price summary (FR-12).

```

computeSummary(items):
  subtotal = sum(item.unitPrice * item.quantity)           # integers
  tax      = round(subtotal * TAX_RATE_PERCENT / 100)      # rounded half up to a whole minor unit
  shipping = (items is empty) ? 0 : SHIPPING_FLAT_FEE
  total    = subtotal + tax + shipping

```

Reservation at checkout (FR-30, FR-11, FR-12).

```

createOrder(userId, addressInput):
  cart    = cartService.getValidatedCart(userId)           # 400 if the cart is empty
  address = resolve(addressInput)                          # saved or entered; 400 if invalid
  releaseOpenCheckout(userId)                             # releases the customer's earlier unpaid order
  session.startTransaction()
  try:
    inventory.reserve(cart.items, session)
    order = orders.insert({ status: PendingPayment, items: snapshots, totals, address snapshot,
                           reservationExpiresAt: now + RESERVATION_MINUTES }, session)
    session.commitTransaction()
  catch INSUFFICIENT_STOCK:
    session.abortTransaction()
    throw INSUFFICIENT_STOCK (409)                        # nothing stays reserved
  try:
    payment = paymentService.createPaymentSession(order)
  catch GatewayError:
    releaseOrder(order, "payment_failed")
    throw PAYMENT_GATEWAY_ERROR (502)
  return { order, payment }

```

Reserve, finalise, release and restore (FR-30, FR-16). Available units are stock - reserved.

```

reserve(items, session):
  for item in items:
    r = books.updateOne({ _id: item.bookId, status: "approved",
                        $expr: { $gte: [ { $subtract: ["$stock", "$reserved"] }, item.quantity ] } },
                        { $inc: { reserved: +item.quantity } }, { session })
    if r.modifiedCount != 1: throw AppError("INSUFFICIENT_STOCK", 409)

finalizeReservation(items, session):                        # at payment confirmation
  for item in items:
    r = books.updateOne({ _id: item.bookId, reserved: { $gte: item.quantity } },
                        { $inc: { stock: -item.quantity, reserved: -item.quantity } }, { session })
    if r.modifiedCount != 1: throw AppError("INTERNAL_ERROR", 500) # not expected; aborts the transaction
  collect books where stock - reserved <= lowStockThreshold # alerts are sent after the commit

releaseReservation(items, session):                        # payment failed, abandoned or expired
  for item in items:
    books.updateOne({ _id: item.bookId, reserved: { $gte: item.quantity } }, { $inc: { reserved: -item.quantity } }, { session })

restore(items, session):                                  # a Placed or Packed order is cancelled
  for item in items:
    books.updateOne({ _id: item.bookId }, { $inc: { stock: +item.quantity } }, { session })

```

Because the reservation filter contains `stock - reserved >= quantity`, the check and the change are one atomic step on the book document, so two buyers can never both reserve the last copy. A failure on any line aborts the transaction, so the lines reserved before it are rolled back. A schema validator also keeps `reserved <= stock` and `stock >= 0`.

Payment confirmation (FR-13, FR-14, FR-30).

```

confirmPayment(userId, orderId, paymentRef):
    order = orders.findOne({ _id: orderId, userId })                # 404 if not found
    if order.status is Placed or later: return order                # idempotent
    if order is Cancelled, or PaymentFailed for a reason other than "expired": throw ORDER_NOT_PAYABLE (409)
    if not paymentService.verifyPayment(order, paymentRef): throw PAYMENT_VERIFICATION_FAILED (400)
    session.startTransaction()
    try:
        if order.status == PaymentFailed:                          # the reservation had expired: reserve
again
            inventory.reserve(order.items, session)
            inventory.finalizeReservation(order.items, session)
            order.status = Placed; each item Placed; order.placedAt = now; payment.status = succeeded
            cart.clear(userId, session)
            session.commitTransaction()
    catch INSUFFICIENT_STOCK:                                       # late payment and the stock is gone
        session.abortTransaction()
        paymentService.refund(payment, order.total)
        order.failureReason = "stock_unavailable_refunded"
        throw ORDER_FAILED_REFUNDED (409)
    notification.sendOrderConfirmation(order, user)                 # after the commit; never fails the order
    return order

```

Cancellation (FR-16).

```

cancelOrder(userId, orderId):
    order = orders.findOne({ _id: orderId, userId })                # 404 if not found
    if the order is closed, or any item is Shipped or Delivered: throw ORDER_NOT_CANCELLABLE (409)
    if order.status == PendingPayment:
        releaseOrder(order, "abandoned")                           # reserved decreases; PaymentFailed; no
    refund                                                         # Placed or Packed
    else:
        session.startTransaction()
        inventory.restore(order.items, session)
        order.status = Cancelled; each item Cancelled
        session.commitTransaction()
        paymentService.refund(payment, order.total)                 # after the commit; on error payment =
    refund_pending

```

Expiry sweeper (interface I-27).

```

releaseExpiredReservations():                                     # at start-up and every minute
    for order in orders.find({ status: PendingPayment, reservationExpiresAt: { $lt: now } }).limit(100):
        releaseOrder(order, "expired")                             # its own transaction; repeating it is harmless

```

Login and logout (FR-02, VFR-07).

```

login(email, password):
    user = users.findByEmail(email)
    if not user: throw INVALID_CREDENTIALS
    if user.lockUntil > now: throw ACCOUNT_LOCKED (423, Retry-After)
    if not bcrypt.compare(password, user.passwordHash):
        user.failedLoginCount += 1
        if user.failedLoginCount >= 5: user.lockUntil = now + 15 minutes; user.failedLoginCount = 0
        throw INVALID_CREDENTIALS
    if user.status == suspended: throw ACCOUNT_SUSPENDED
    if user.status == pending_verification: throw EMAIL_NOT_VERIFIED
    user.failedLoginCount = 0
    return signToken({ sub: user.id, tv: user.tokenVersion }, expiresIn: 60 minutes)

```

Token check on every protected request (authGuard).

```

authGuard(req):
    payload = verifyToken(cookie.session)                          # signature and expiry; else 401
    user = users.findById(payload.sub)
    if not user or user.tokenVersion != payload.tv: throw UNAUTHENTICATED (401)
    if user.status == suspended: throw ACCOUNT_SUSPENDED (403)
    req.user = { id, roles }

```

Search (FR-05, FR-06). Search uses derived fields and a trigram index. The algorithm, fields and indexes are in Section 4.4.

Review eligibility (FR-17). A customer may review a book only if an order of that customer contains an item for that book with `fulfilmentStatus = Delivered`. A unique index on { `bookId`, `userId` } guarantees one review per customer per book even under concurrent requests.

4.4 Search Design

This section defines the primary search design for FR-05 (partial, case-insensitive search on title, author and ISBN), FR-06 (filters and sorting) and VFR-02 (search and browse API at most 500 ms at the 95th percentile). The design is index-based from the start; the contingency options at the end are only for the case that performance testing shows a problem.

4.4.1 Derived search fields

When a listing is created or its title, author or ISBN changes, the Book Management module asks the Catalog module for the search fields (interface I-28) and stores them on the book. Clients never send them and the API never returns them.

Field	Content	Used for
<code>titleLower</code>	Title in lower case with accents removed	Prefix search of 1 or 2 characters
<code>authorLower</code>	Author in lower case with accents removed	Prefix search of 1 or 2 characters
<code>searchText</code>	Normalised title, author and the ISBN digits, joined by single spaces	Exact confirmation of a substring match
<code>searchTrigrams</code>	The distinct 3-character substrings of <code>searchText</code>	Index lookup of substring matches

Normalisation (used for stored text and for the query alike): convert to lower case, remove accents (Unicode decomposition), trim and collapse repeated spaces. Example: for the book "Harry Potter and the Philosopher's Stone" by "J. K. Rowling" with ISBN 9780747532699, `searchText` is `harry potter and the philosopher's stone j. k. rowling 9780747532699` and `searchTrigrams` contains `har`, `arr`, `rry`, `ry`, `y p`, and so on.

4.4.2 Query behaviour

```
search(q):
  t = normalise(q.text)
  if length(t) >= 3:
    grams = distinct 3-character substrings of t
    filter = { status: "approved", searchTrigrams: { $all: grams }, searchText: /escapeRegex(t)/ }
  else:
    p = escapeRegex(t)
    filter = { status: "approved", $or: [ { titleLower: /^p/ }, { authorLower: /^p/ }, { isbn: /^p/ } ] }
  add category, price range, minRating and language conditions to filter
  sort by newest, price or rating; skip (page - 1) * 20; limit 20
  totalItems = count of documents matching filter
```

How the three searched fields are supported:

Search target	Support
Title	Substring anywhere (3 or more characters) through the trigram index; prefix for 1 or 2 characters
Author	Same as title
ISBN	Included in <code>searchText</code> as digits, so a fragment of 3 or more digits matches anywhere; a prefix of 1 or 2 characters uses the <code>isbn</code> index. Hyphens and spaces in the typed ISBN are removed first

The `$all` condition on the multikey index { `status`, `searchTrigrams` } returns only the books that contain every 3-character piece of the query, which for a realistic query is a handful of documents. The exact substring check on `searchText` then removes false candidates (books that contain all the pieces but not in the same order). The regular expression is therefore evaluated on a small candidate set, not on the catalogue. The text is escaped before use, so characters such as `.`, `*` and `$` are matched literally (VFR-06).

Behaviour to note: a multi-word query is matched as one phrase, so "potter harry" does not match "Harry Potter"; stop words are not removed; accents and case are ignored.

4.4.3 Indexes

Index (collection books)	Purpose
{ status: 1, searchTrigrams: 1 }	Substring search of 3 or more characters
{ status: 1, titleLower: 1 }, { status: 1, authorLower: 1 }	Prefix search of 1 or 2 characters
{ isbn: 1 }	Prefix search by ISBN
{ status: 1, categoryId: 1, price: 1 }, { status: 1, price: 1 }	Category and price filters and price sorting
{ status: 1, avgRating: -1 }	Rating filter and sort
{ status: 1, createdAt: -1 }	Browsing newest first

Storage and write cost: a book with about 100 characters of searchText has at most 98 trigrams, so 10,000 books need fewer than one million index entries. The fields are recomputed only when a listing is created or its title, author or ISBN changes.

4.4.4 Why it is expected to meet VFR-02

Every search and every browse request is answered from an index and returns a page of 20 documents; the count uses the same filter. No request scans the catalogue or evaluates a pattern over it. This is a design argument, not a measurement: PT-01 (1,000 books) and PT-06 (10,000 books) in the Validation Specification measure it, and the query plans are checked to confirm the indexes are used.

4.4.5 Contingency options

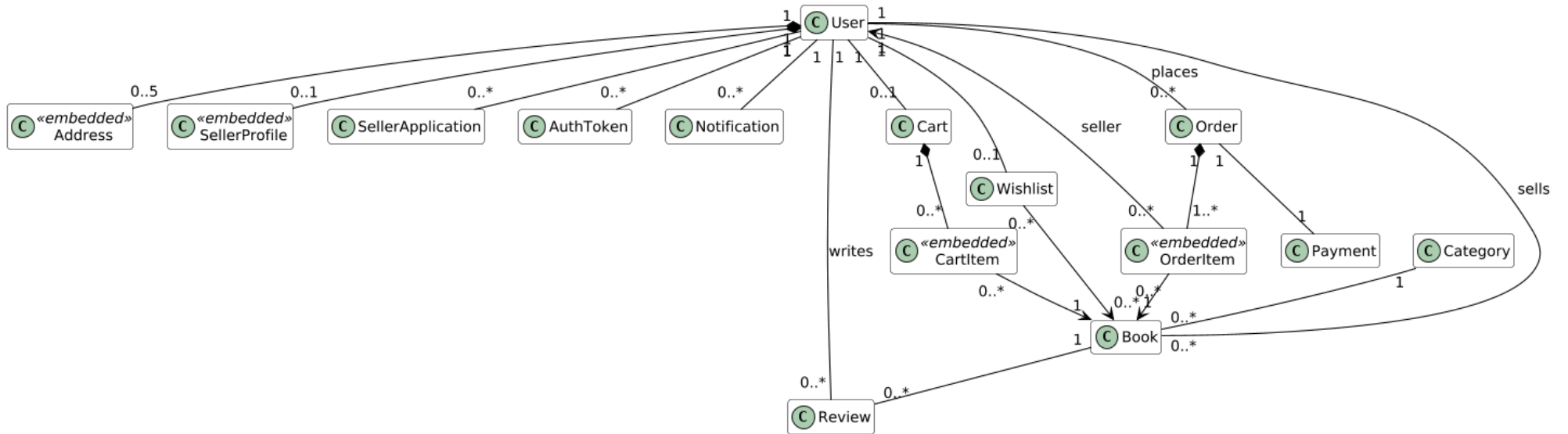
Used only if PT-01 or PT-06 shows VFR-02 is missed: cache the totalItems count for a short time; limit the number of trigrams taken from very long queries; add an index for the most common filter and sort pair; use a MongoDB text index for whole-word queries; or use a hosted search feature if the chosen database tier offers one.

5. UML Models

5.1 Domain class diagrams

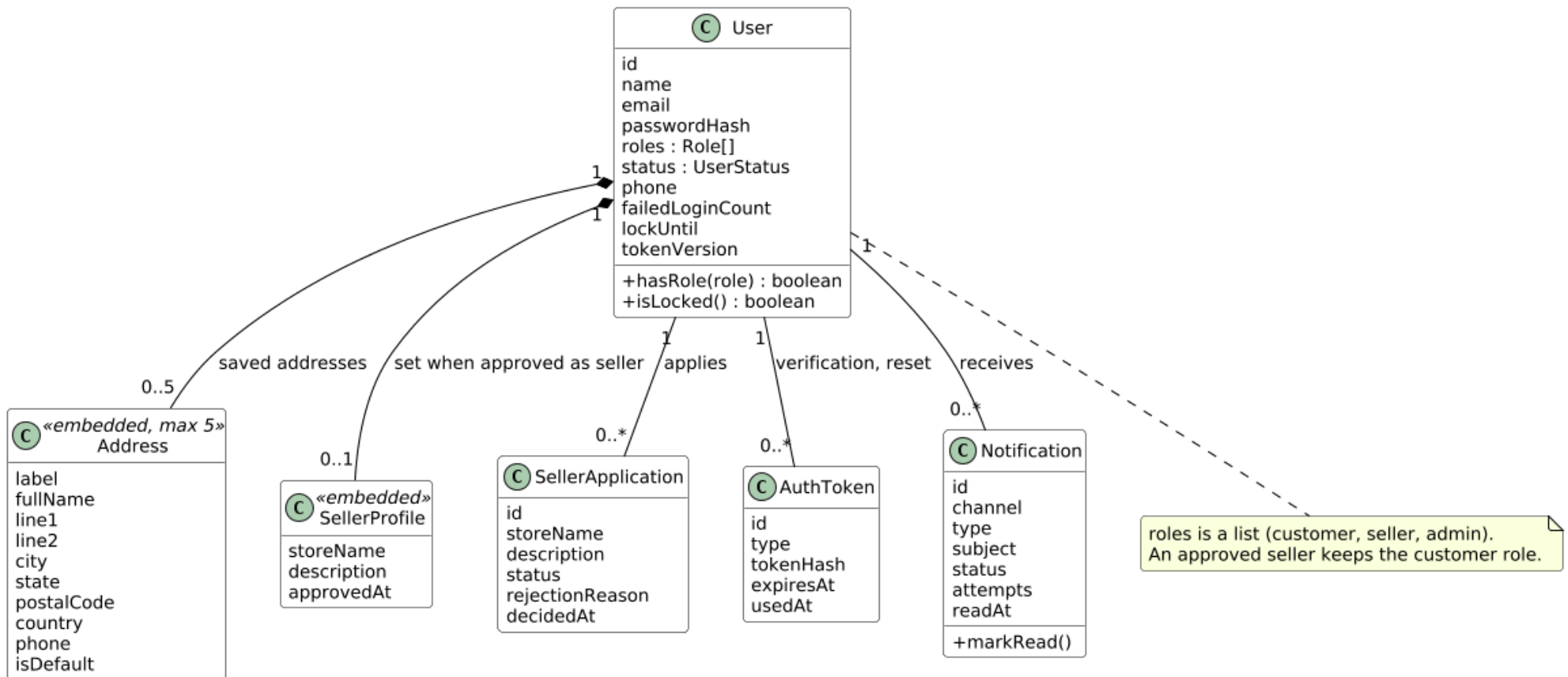
The domain model has 15 classes. A single diagram with all attributes cannot be read at A4 size, so it is shown as an overview of all classes and relationships, followed by three diagrams that add the attributes of one part of the model each. The three parts follow the ownership of the data: accounts, catalog, and purchasing.

Online Bookstore: Domain Overview (classes and relationships)



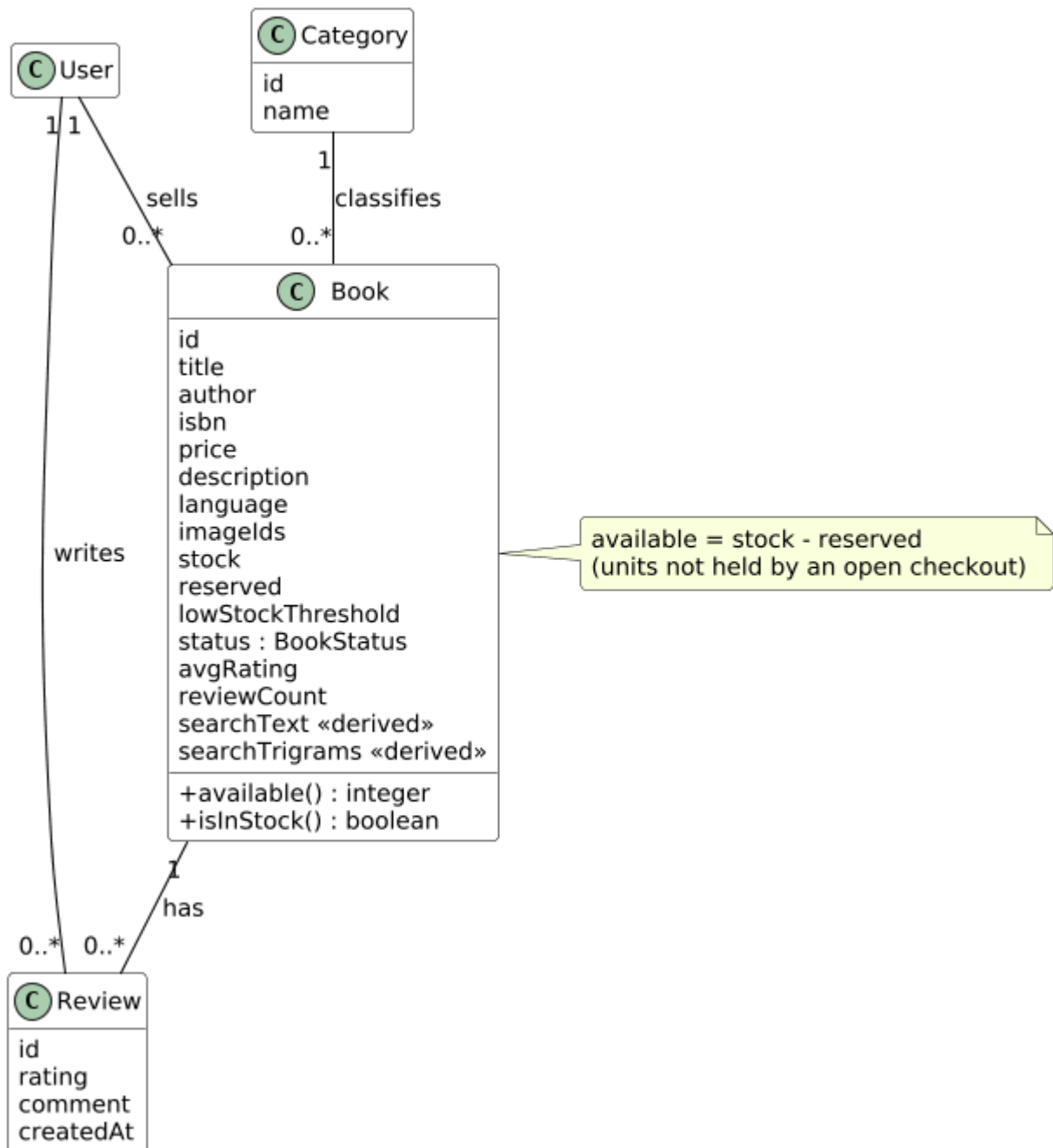
Domain overview: every class and relationship, without attributes. Classes marked «embedded» are stored inside their parent document.

Domain Classes 1 of 3: Accounts and Communication



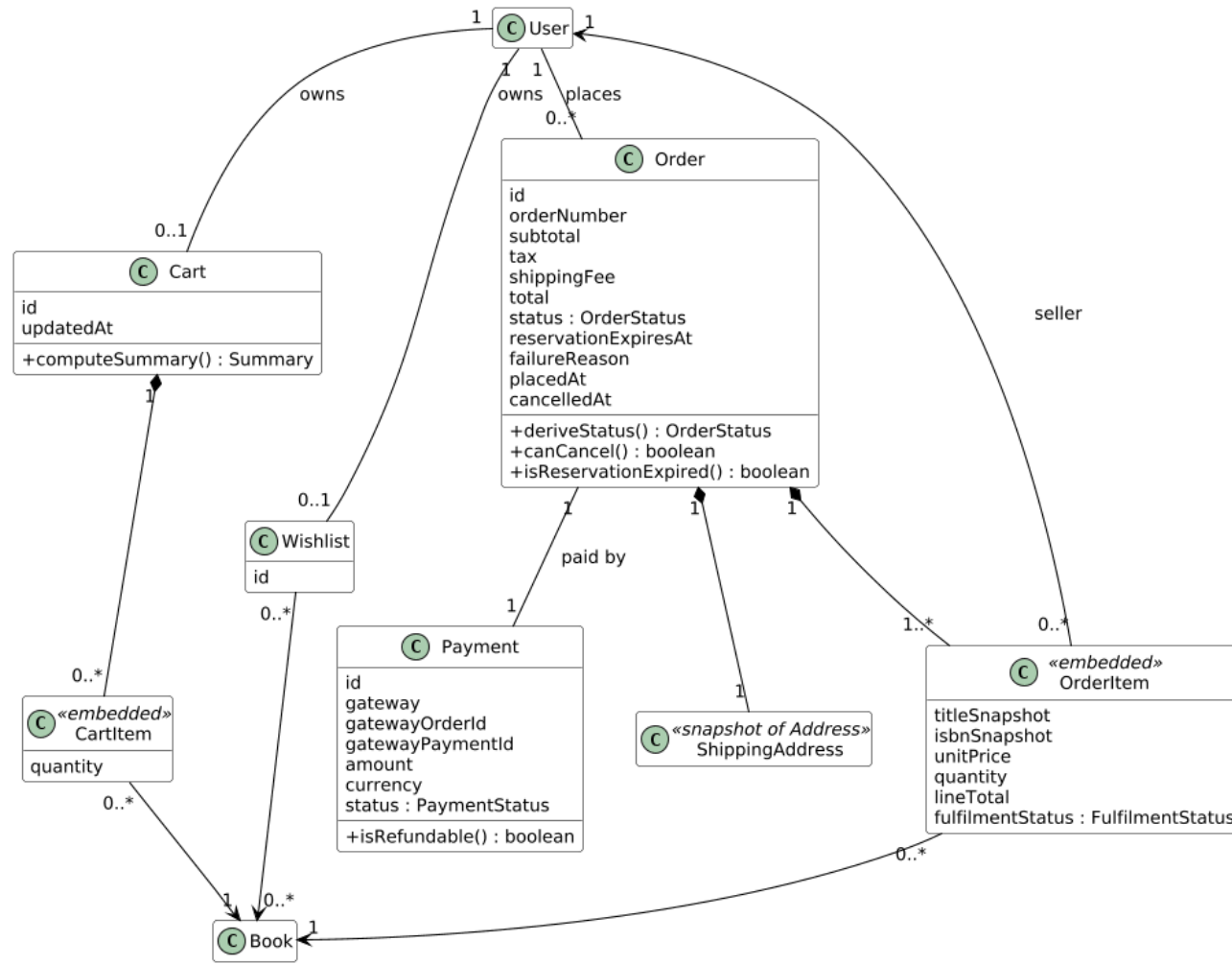
Accounts and communication: User, Address, SellerProfile, SellerApplication, AuthToken and Notification.

Domain Classes 2 of 3: Catalog and Reviews



Catalog and reviews: Category, Book and Review. A Book has stock and reserved; available units are stock minus reserved.

Domain Classes 3 of 3: Cart, Wishlist, Orders and Payment



Purchasing: Cart, Wishlist, Order and Payment, with the reservation expiry and failure reason on Order.

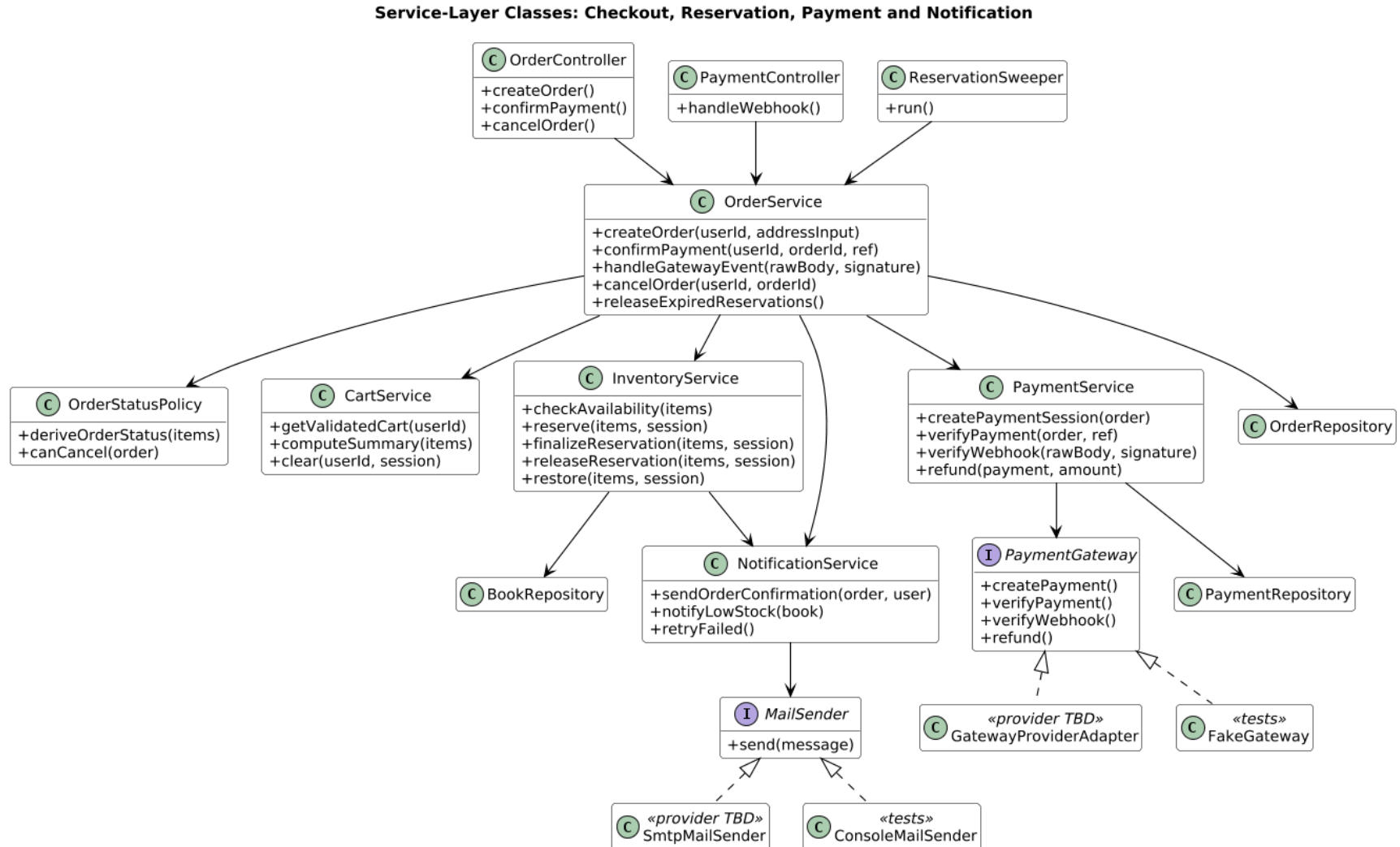
Notes on the model:

- There are no Customer, Seller or Administrator subclasses. A user has a list of roles, because one person can be both Customer and Seller (D-04). Inheritance would not describe that correctly, and there is no rule that excludes Customer once a user is a Seller.
- SellerProfile is a value object that exists only for sellers and carries the store name.
- Address is embedded in the user (at most 5) and copied into the order as a snapshot.
- OrderItem copies title, ISBN and price so order history never changes when a listing is edited or removed.
- Payment holds gateway references only (VFR-08).
- The derived search fields of Book (titleLower, authorLower, searchText, searchTrigrams) exist in the database but are not part of the domain model; Section 4.4 describes them.

Allowed values of the enumerated attributes:

Attribute	Allowed values
User.roles	customer, seller, admin (a list; an approved seller has customer and seller)
User.status	pending_verification, active, suspended
Book.status	pending, approved, rejected, removed
OrderItem.fulfilmentStatus	Placed, Packed, Shipped, Delivered, Cancelled
Order.status	PendingPayment, PaymentFailed, Placed, Packed, Shipped, Delivered, Cancelled
Order.failureReason	payment_failed, abandoned, expired, stock_unavailable_refunded
Payment.status	created, succeeded, failed, refund_pending, refunded
SellerApplication.status	pending, approved, rejected
Notification.channel / status	email or in_app / queued, sent, failed

5.2 Service-layer class diagram

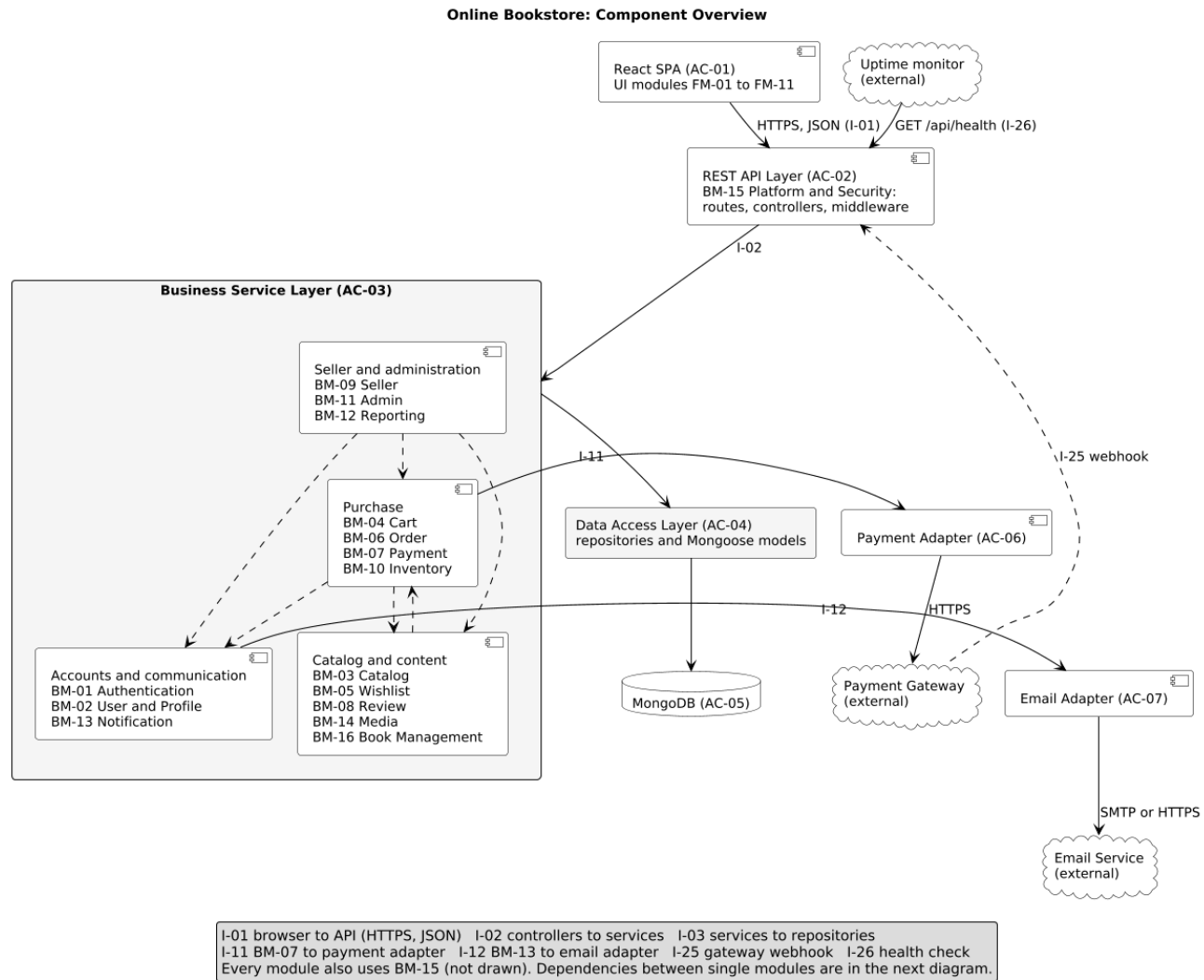


Service-layer classes for checkout, stock reservation, payment and notification. *PaymentGateway* and *MailSender* are interfaces; the provider adapters are TBD and the fake implementations are used in tests.

This diagram shows the one place where a design pattern matters: the adapter pattern around the two external services, so the business logic does not depend on a provider.

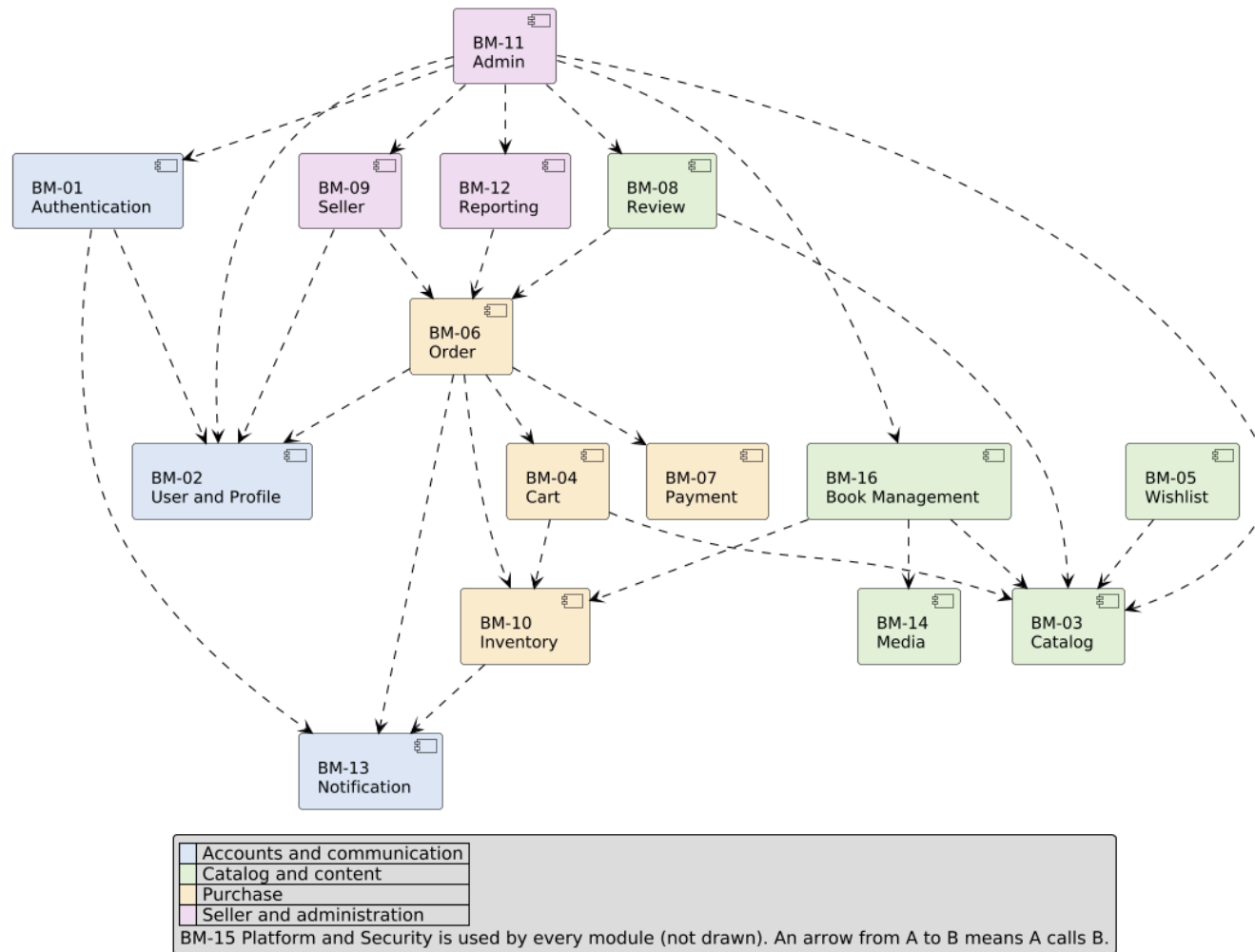
5.3 Component diagrams

The overview shows the layers and groups the sixteen backend modules by area. The second diagram shows the dependencies between single modules.



Component overview. Dashed arrows are dependencies between module groups. Every module also uses BM-15, which is not drawn.

Online Bookstore: Backend Module Dependencies

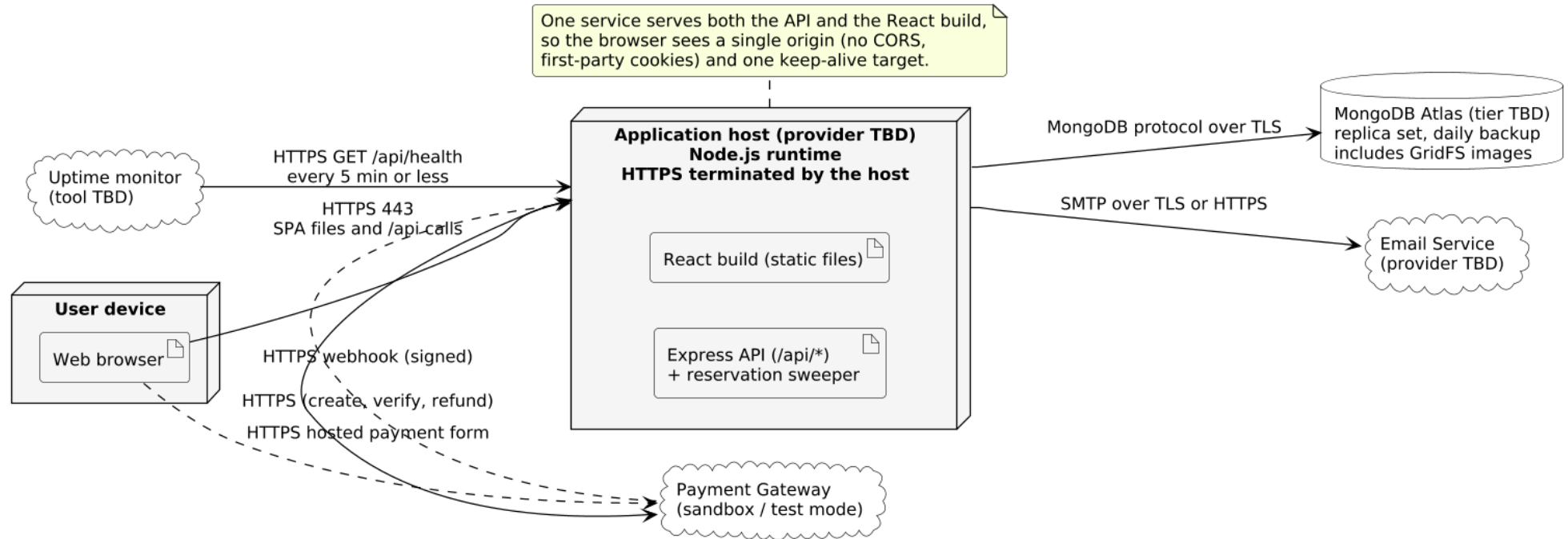


Backend module dependencies. An arrow from A to B means that A calls B through the interface listed in the Architecture document, Section 5. BM-15 is used by every module and is not drawn.

5.4 Deployment diagram

The deployment model is described in the Architecture document, Section 7, and drawn once in `../architecture/deployment-diagram.png` to avoid two copies that could disagree.

Online Bookstore: Deployment Diagram

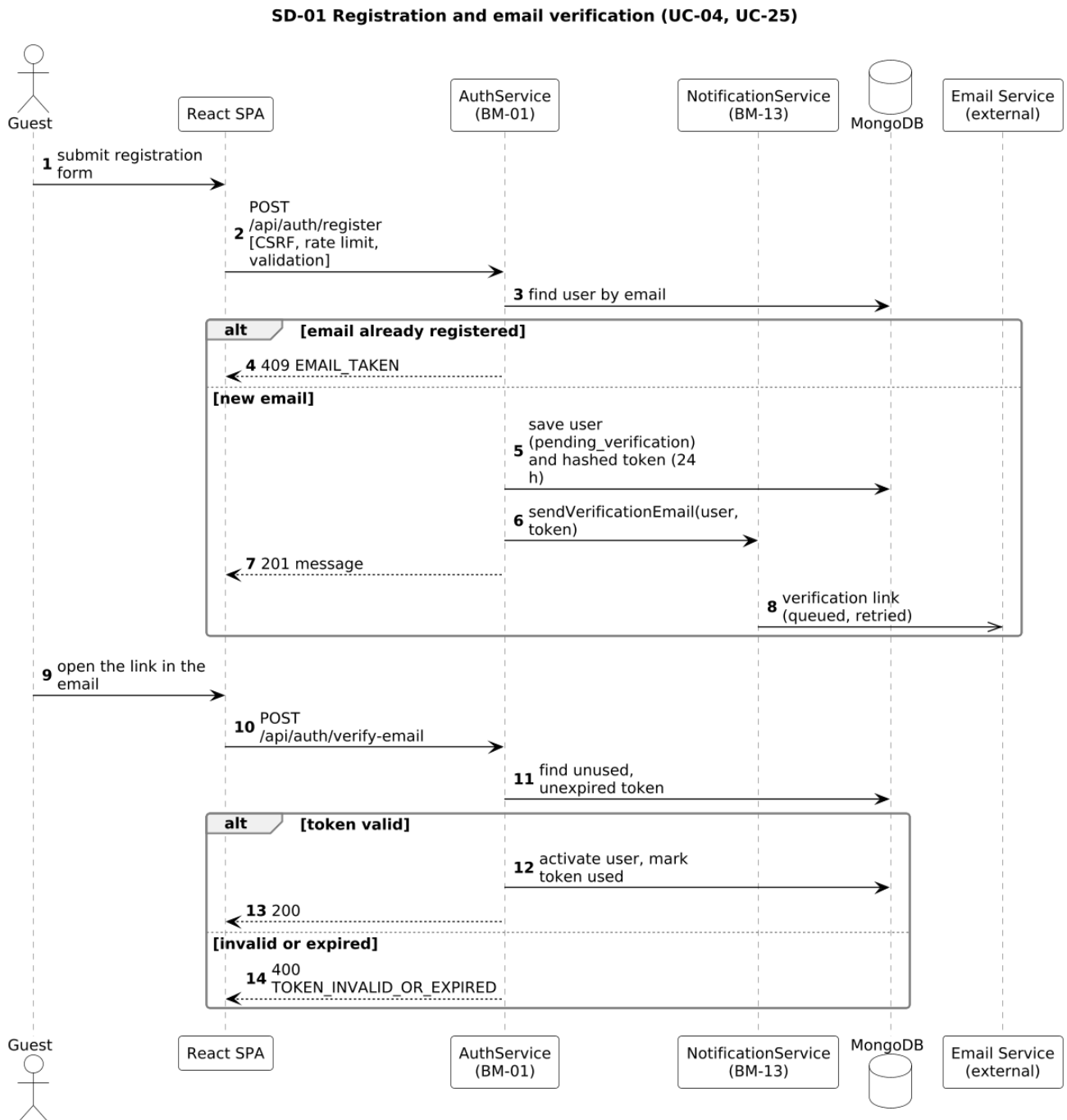


Deployment diagram (the same figure as in the Architecture document).

5.5 Sequence diagrams

Twelve sequence diagrams cover the main flows. Each one corresponds to Phase 1 use cases and uses the endpoints of Section 6. Participant names are the module names of Section 4.

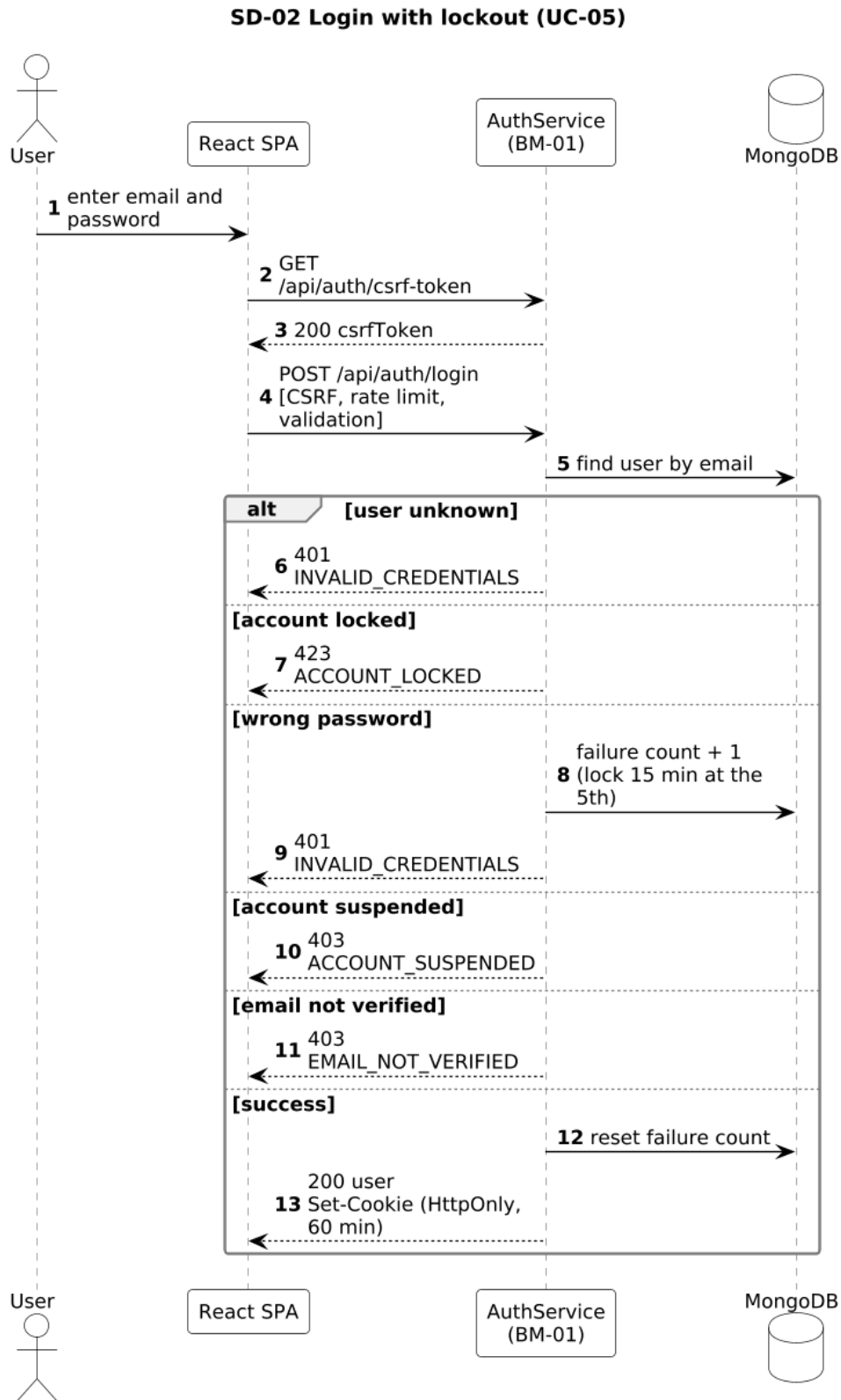
SD-01 Registration and email verification (UC-04, UC-25)



SD-01 Registration and email verification (UC-04, UC-25).

Item	Value
Use cases	UC-04, UC-25
Requirements	FR-01, VFR-04
Endpoints used	API-02, API-03

SD-02 Login with lockout (UC-05)

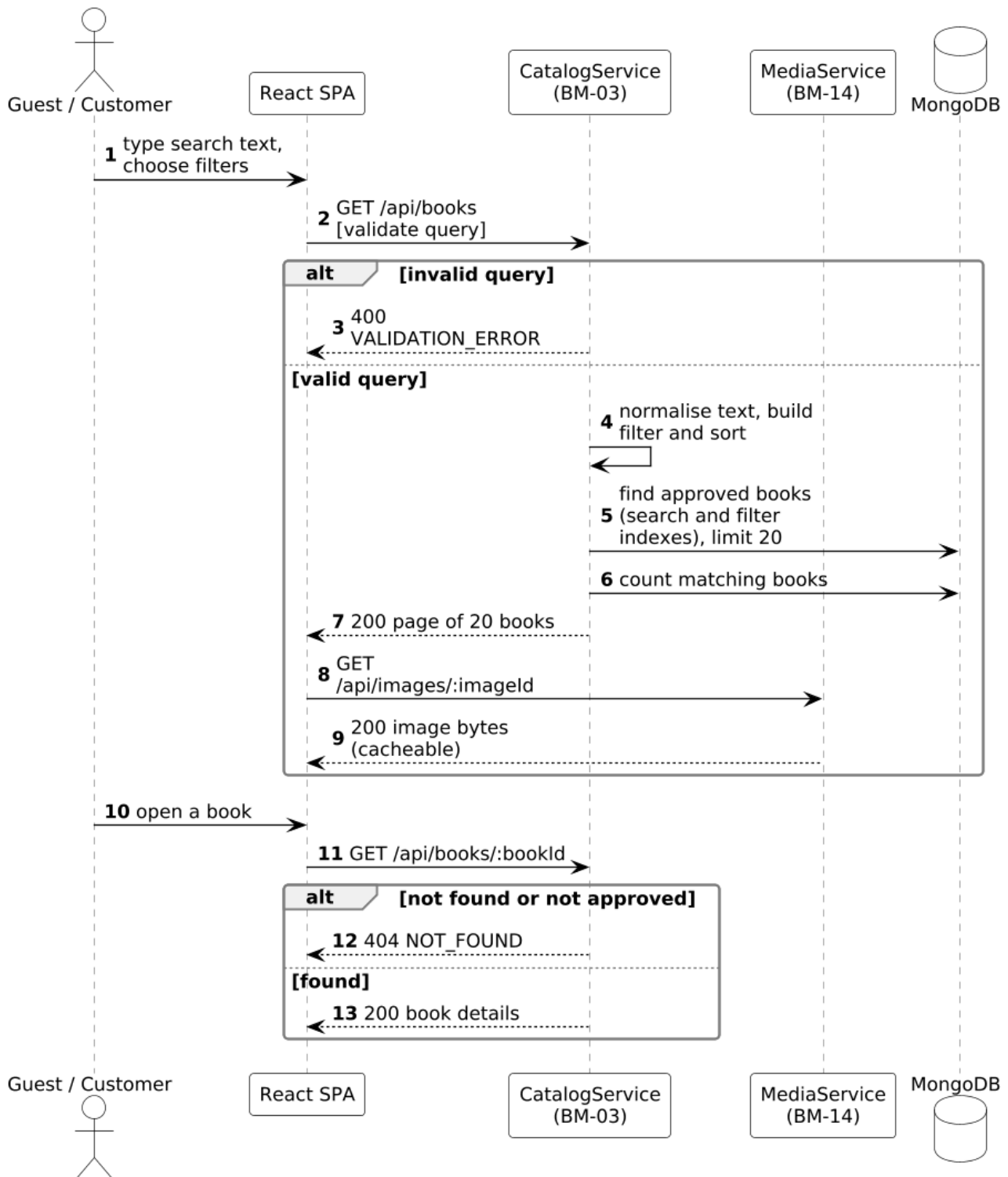


SD-02 Login with lockout (UC-05).

Item	Value
Use cases	UC-05
Requirements	FR-02, FR-24, VFR-06, VFR-07
Endpoints used	API-01, API-05

SD-03 Search and browse books (UC-01, UC-02)

SD-03 Search and browse books (UC-01, UC-02)

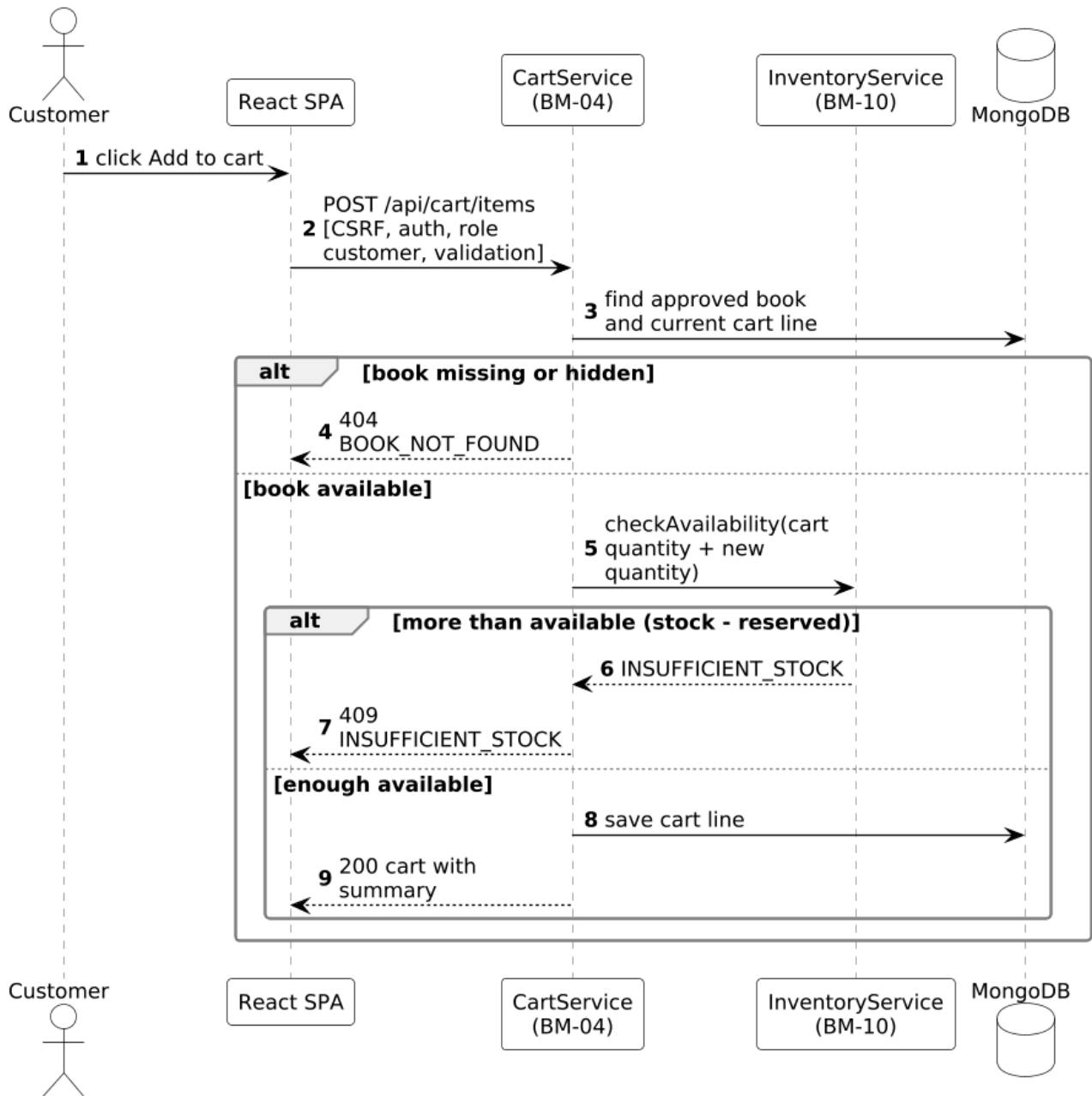


SD-03 Search and browse books (UC-01, UC-02).

Item	Value
Use cases	UC-01, UC-02
Requirements	FR-04, FR-05, FR-06, FR-07, FR-20, VFR-01, VFR-02
Endpoints used	API-15, API-19, API-16

SD-04 Add a book to the cart (UC-06)

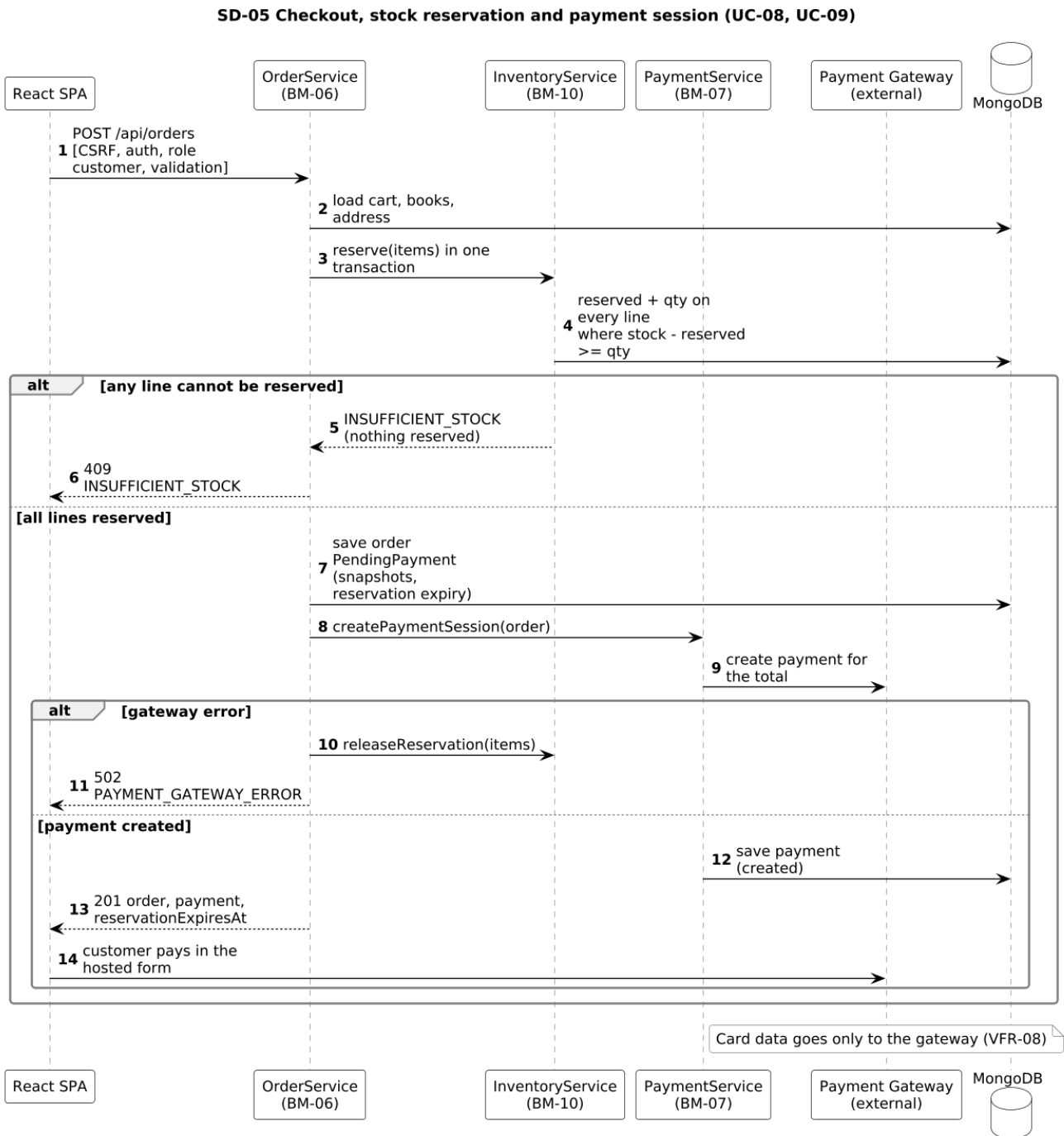
SD-04 Add a book to the cart (UC-06)



SD-04 Add a book to the cart (UC-06).

Item	Value
Use cases	UC-06
Requirements	FR-08, FR-09
Endpoints used	API-21

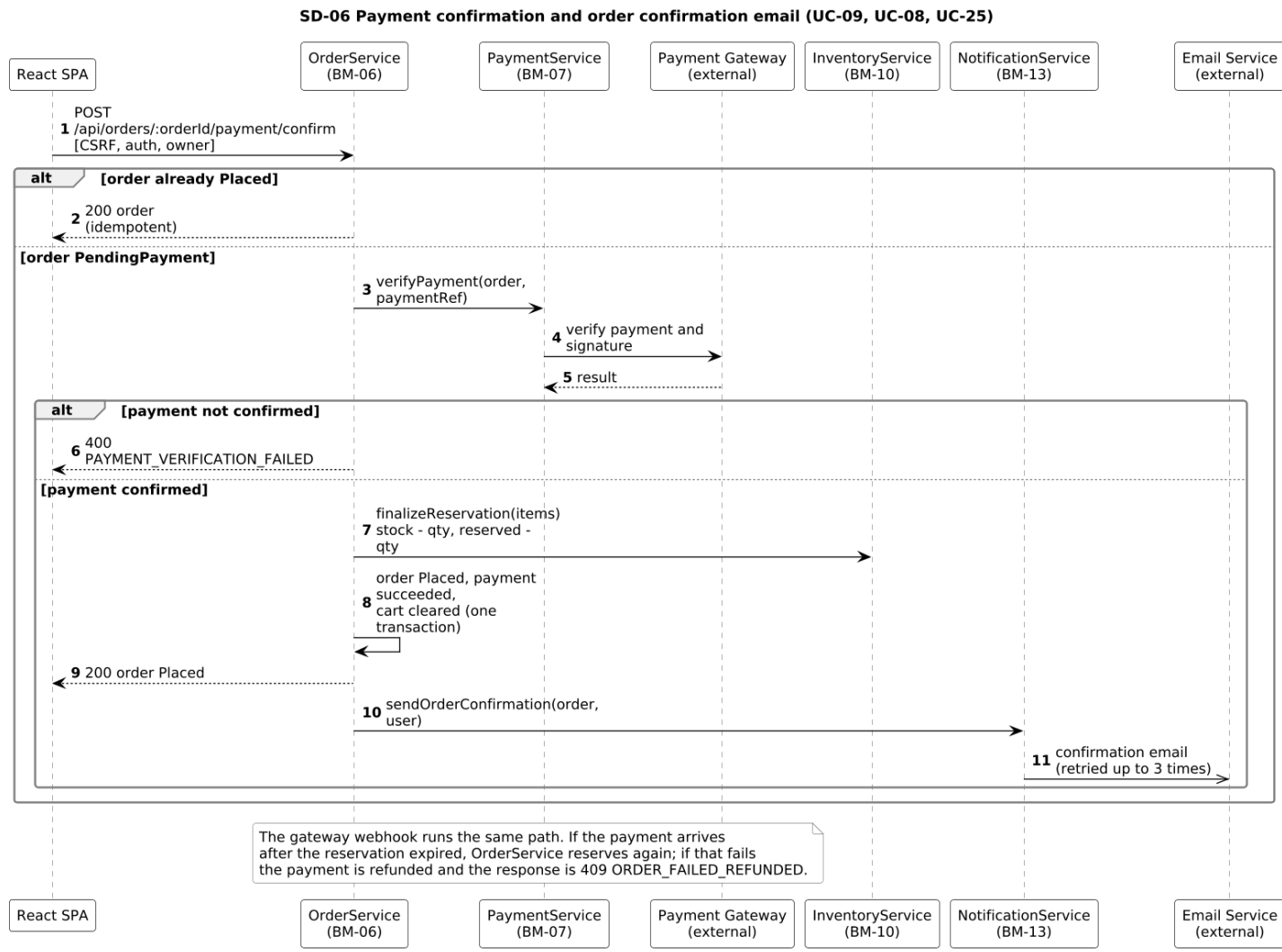
SD-05 Checkout, stock reservation and payment session (UC-08, UC-09)



SD-05 Checkout, stock reservation and payment session (UC-08, UC-09).

Item	Value
Use cases	UC-08, UC-09
Requirements	FR-11, FR-12, FR-13, FR-30, VFR-08, VFR-14
Endpoints used	API-27

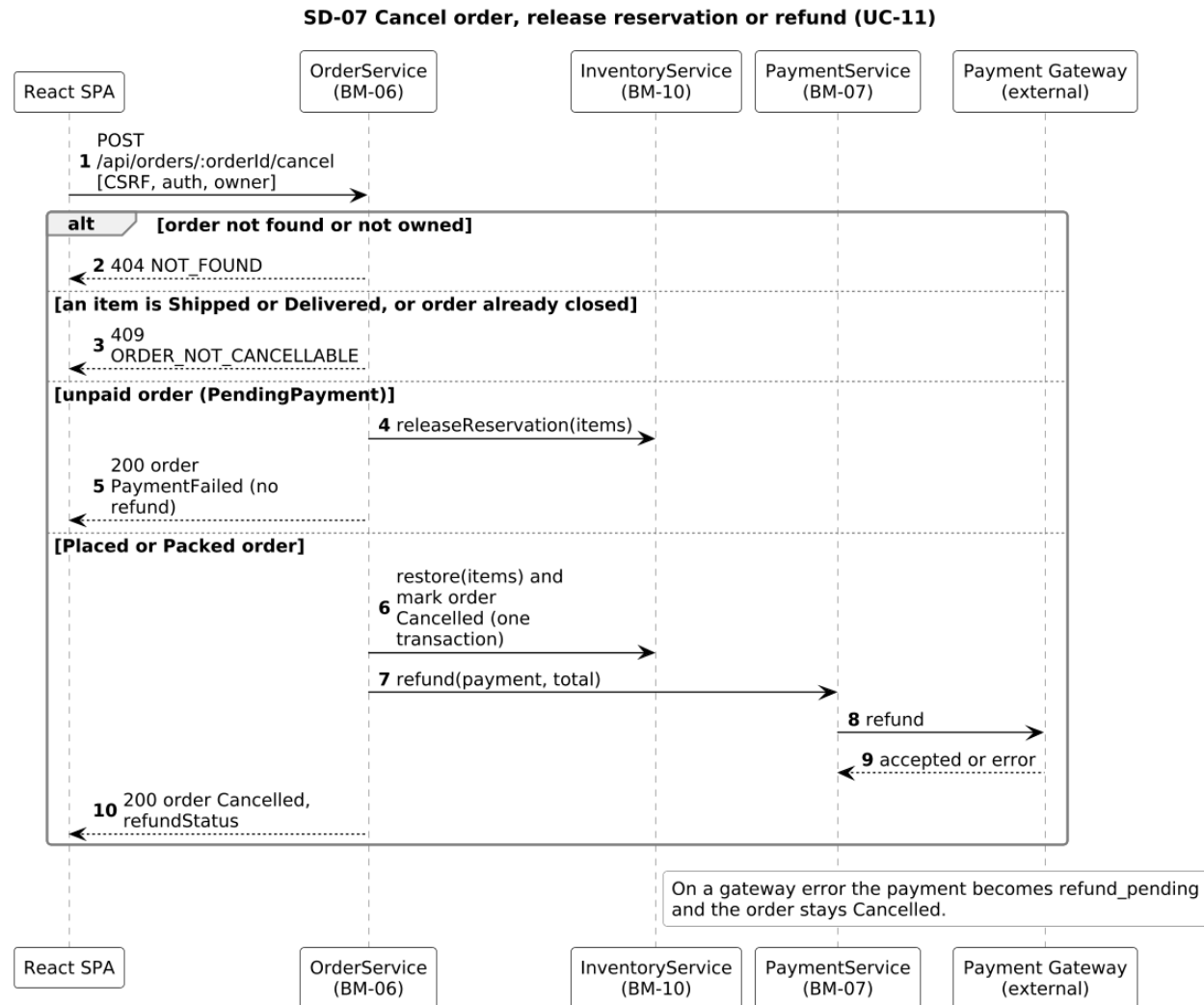
SD-06 Payment confirmation and order confirmation email (UC-09, UC-08, UC-25)



SD-06 Payment confirmation and order confirmation email (UC-09, UC-08, UC-25).

Item	Value
Use cases	UC-09, UC-08, UC-25
Requirements	FR-13, FR-14, FR-30, VFR-08, VFR-14
Endpoints used	API-28

SD-07 Cancel order, release reservation or refund (UC-11)

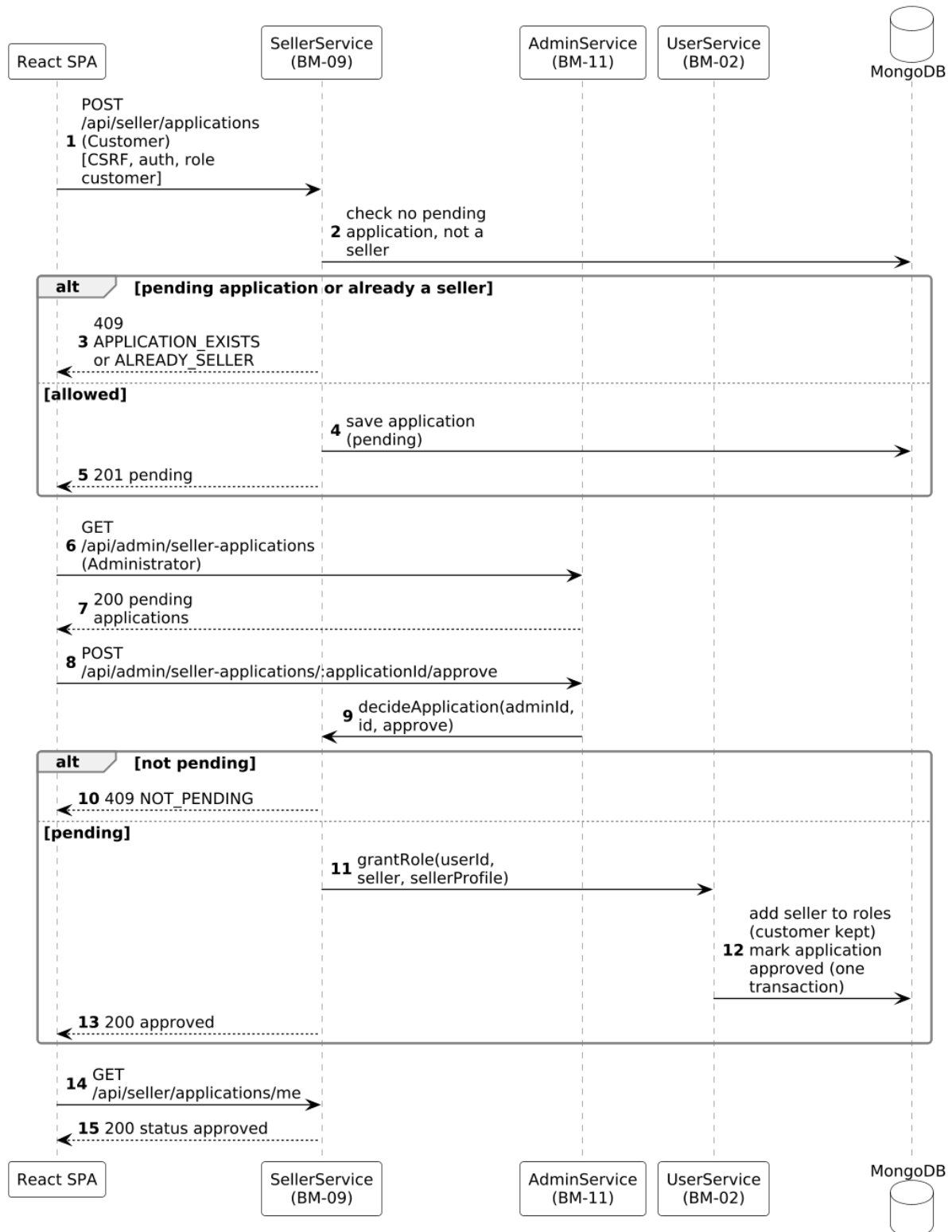


SD-07 Cancel order, release reservation or refund (UC-11).

Item	Value
Use cases	UC-11
Requirements	FR-16, VFR-14
Endpoints used	API-32

SD-08 Seller application and administrator approval (UC-15, UC-21)

SD-08 Seller application and administrator approval (UC-15, UC-21)

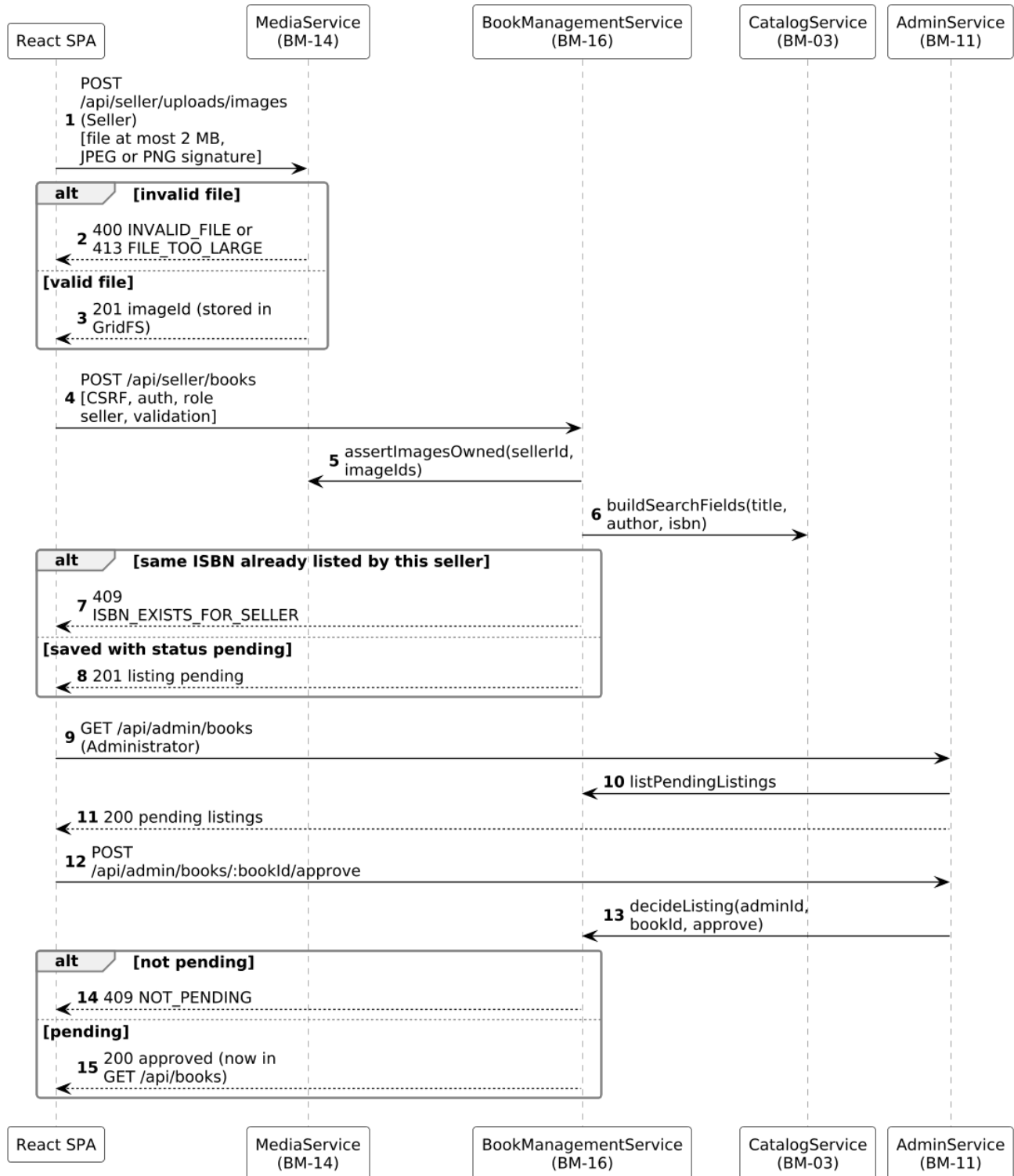


SD-08 Seller application and administrator approval (UC-15, UC-21).

Item	Value
Use cases	UC-15, UC-21
Requirements	FR-19, FR-25
Endpoints used	API-34, API-49, API-50, API-35

SD-09 Seller listing creation and administrator approval (UC-16, UC-21)

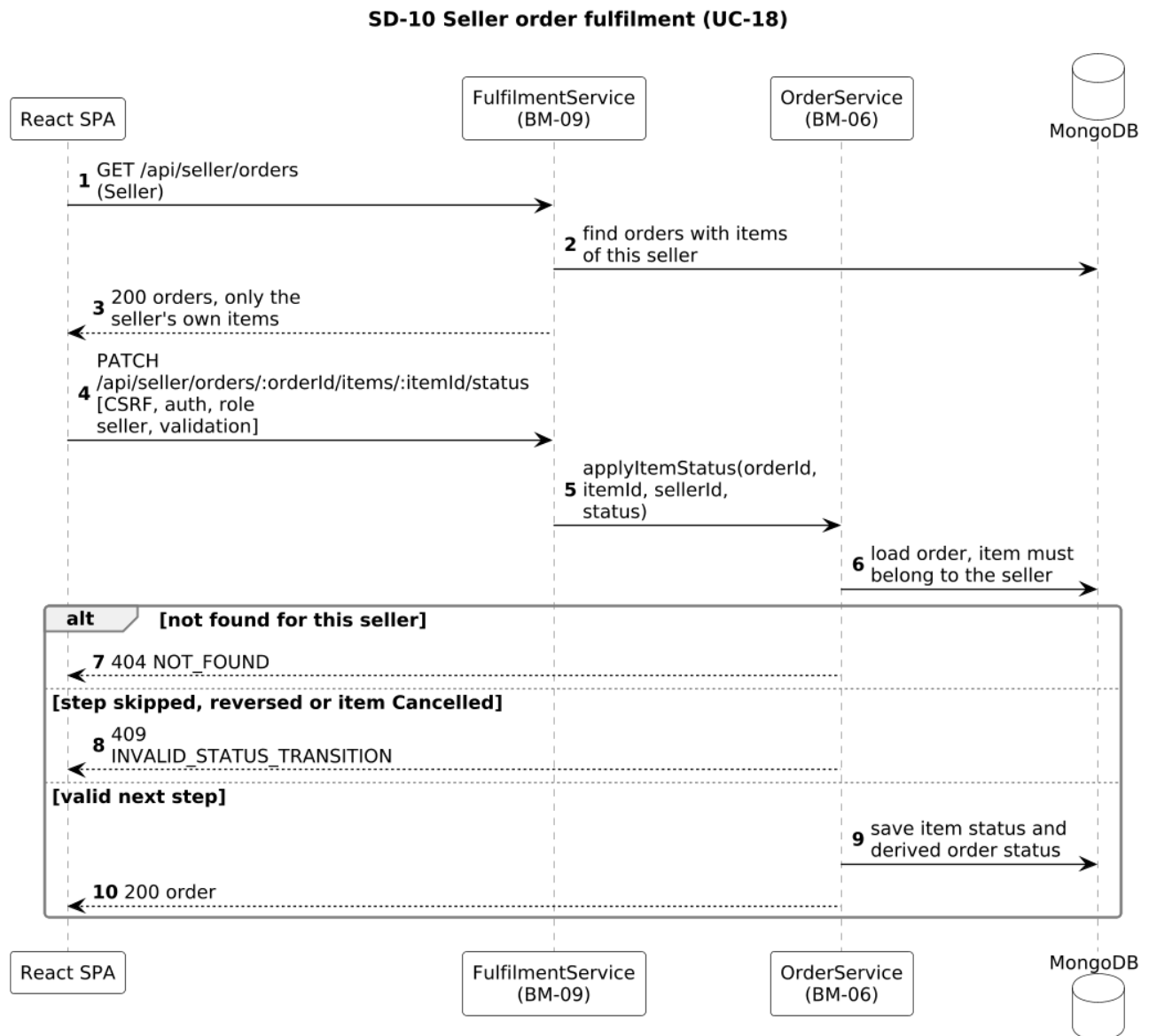
SD-09 Seller listing creation and administrator approval (UC-16, UC-21)



SD-09 Seller listing creation and administrator approval (UC-16, UC-21).

Item	Value
Use cases	UC-16, UC-21
Requirements	FR-04, FR-05, FR-06, FR-20, FR-25, VFR-01, VFR-02
Endpoints used	API-41, API-37, API-52, API-53, API-15

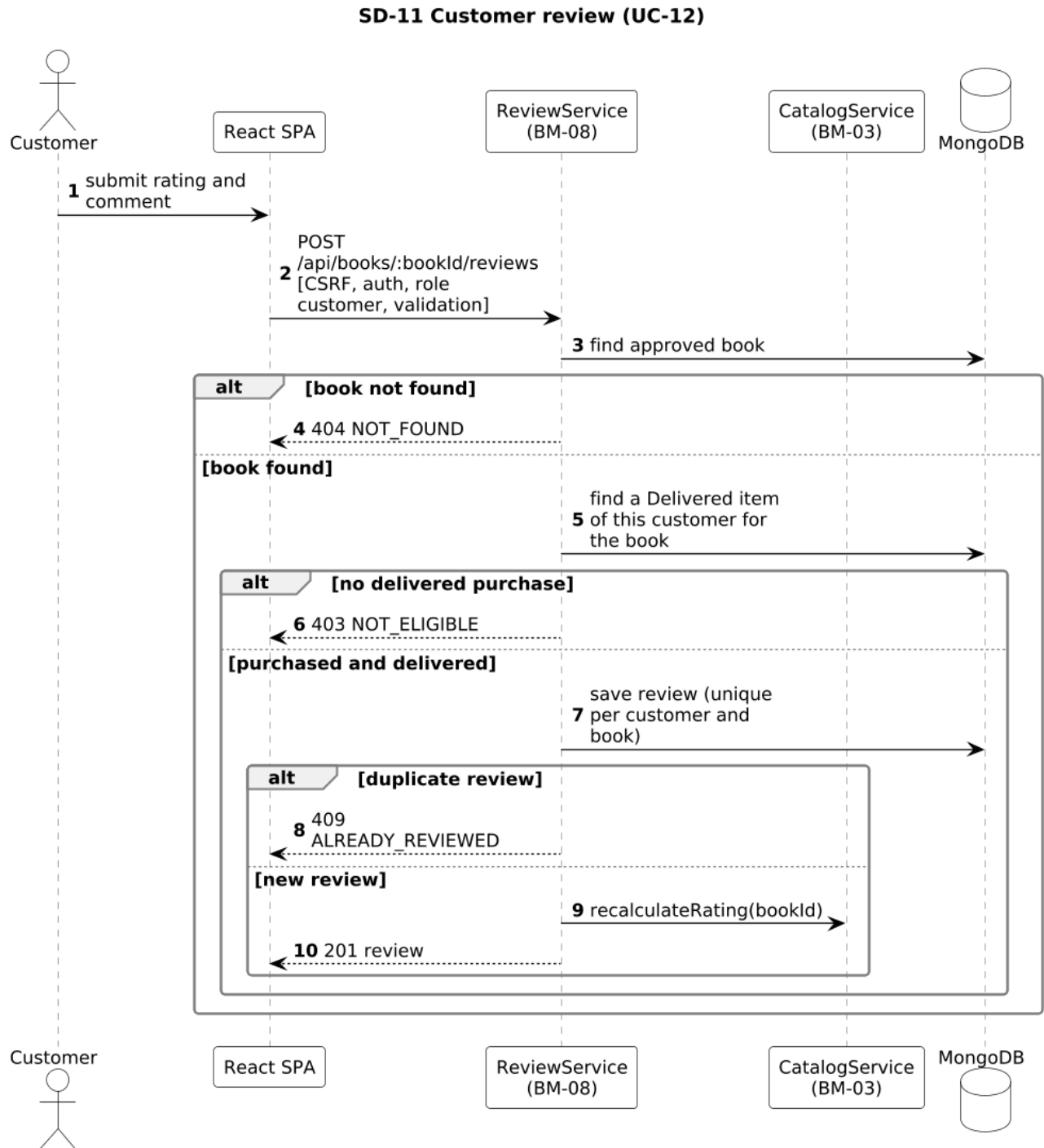
SD-10 Seller order fulfilment (UC-18)



SD-10 Seller order fulfilment (UC-18).

Item	Value
Use cases	UC-18
Requirements	FR-15, FR-22
Endpoints used	API-42, API-43

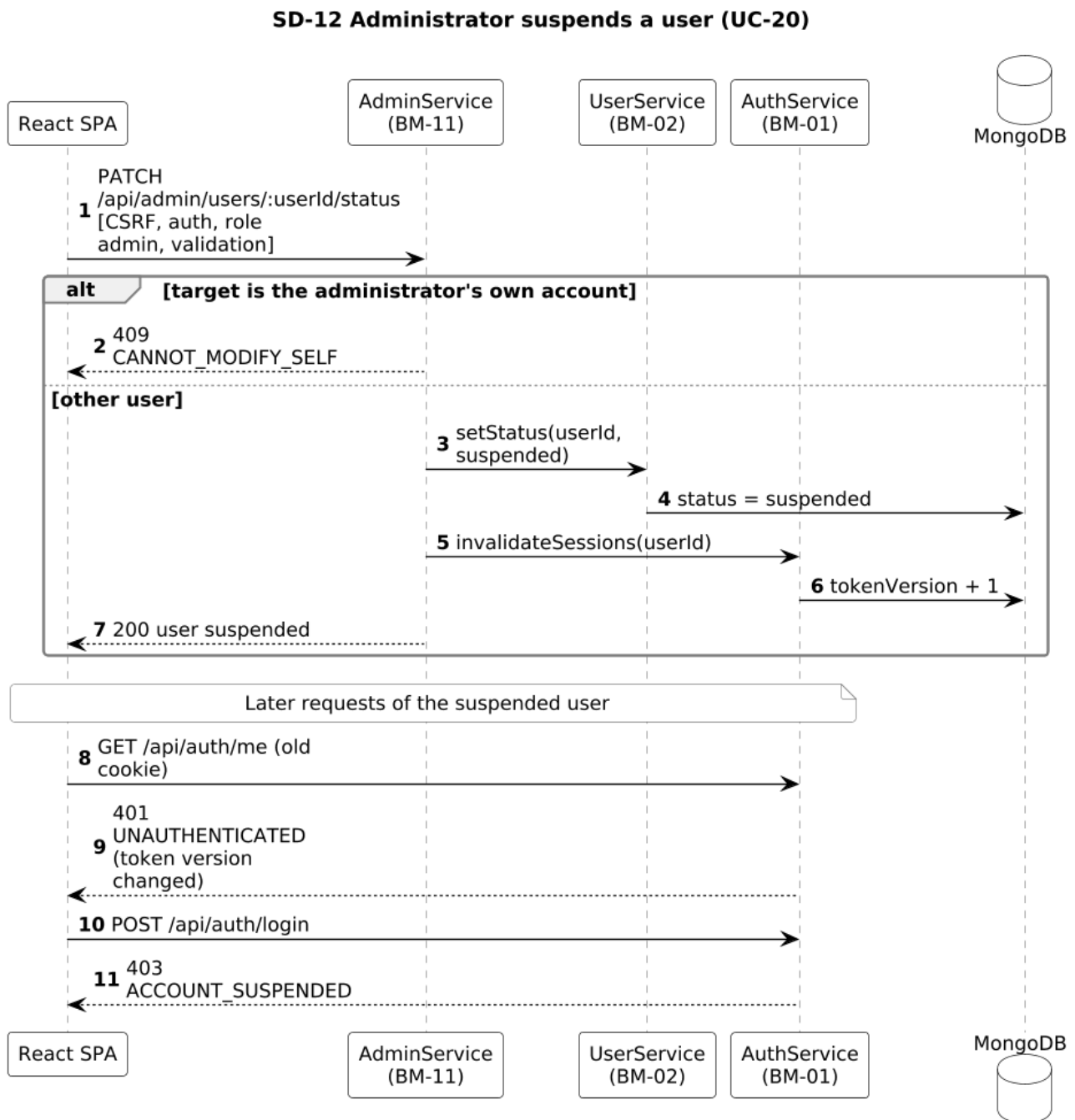
SD-11 Customer review (UC-12)



SD-11 Customer review (UC-12).

Item	Value
Use cases	UC-12
Requirements	FR-17
Endpoints used	API-33

SD-12 Administrator suspends a user (UC-20)

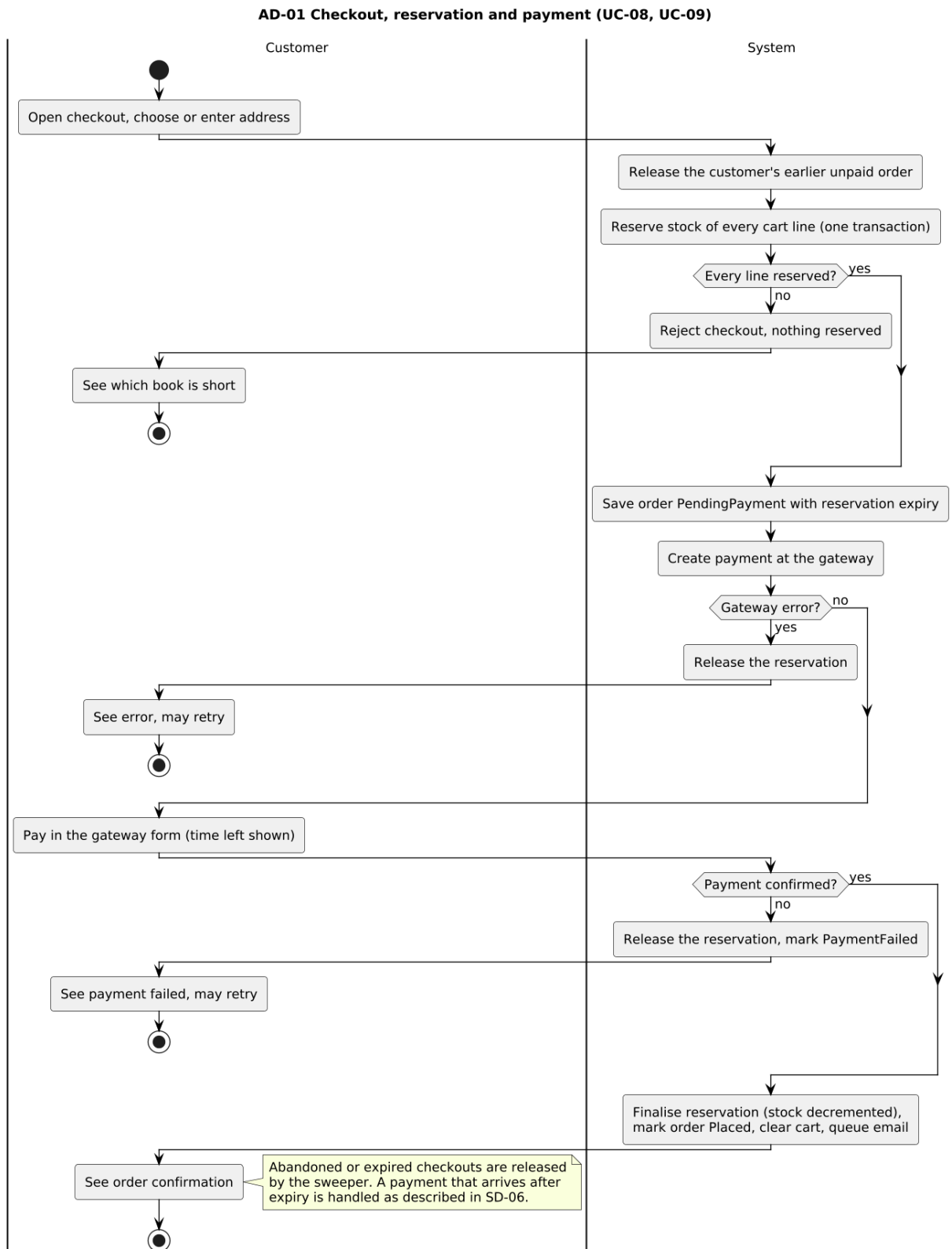


SD-12 Administrator suspends a user (UC-20).

Item	Value
Use cases	UC-20
Requirements	FR-02, FR-24, FR-29, VFR-07
Endpoints used	API-48, API-07, API-05

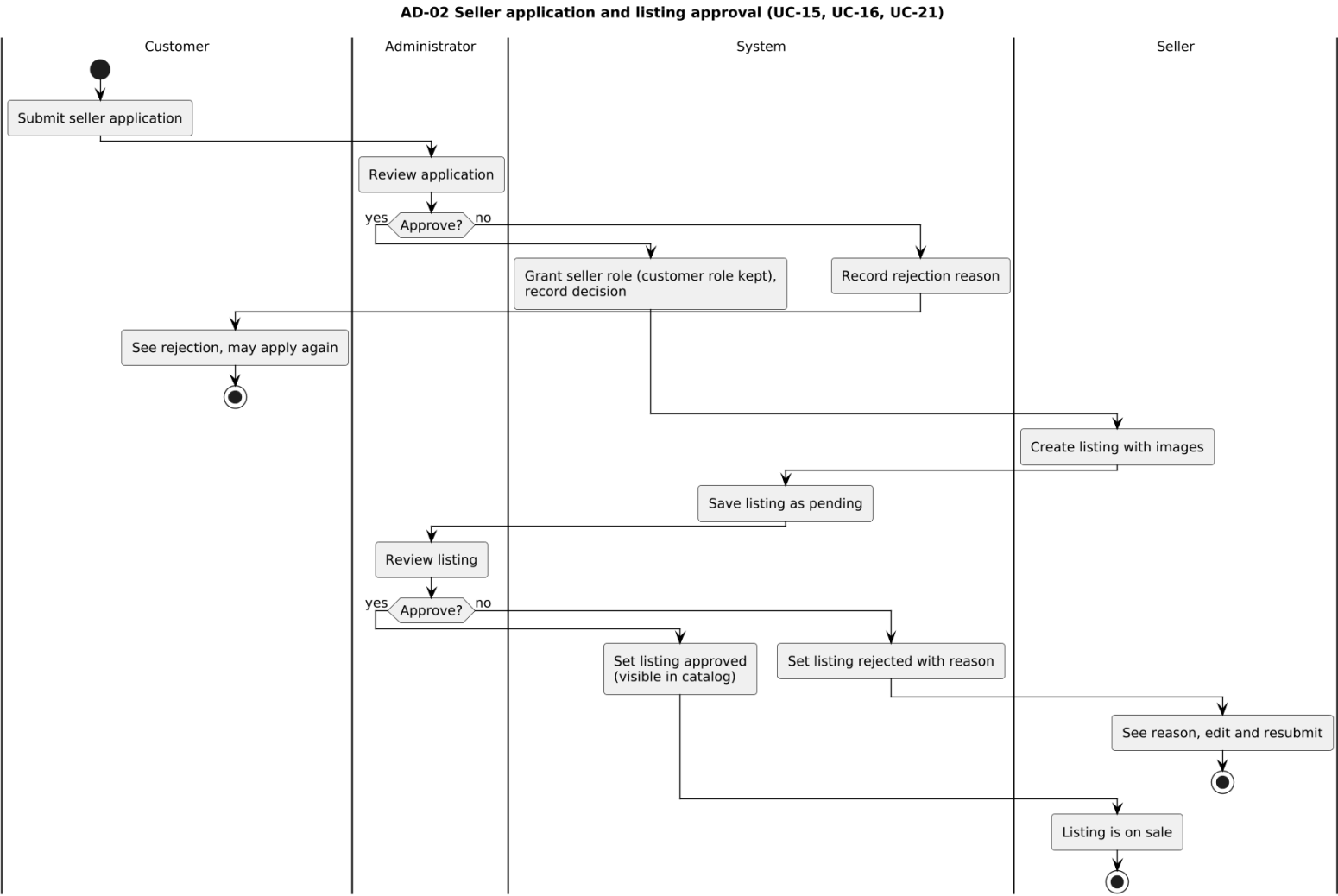
5.6 Activity diagrams

AD-01 Checkout, reservation and payment (UC-08, UC-09)



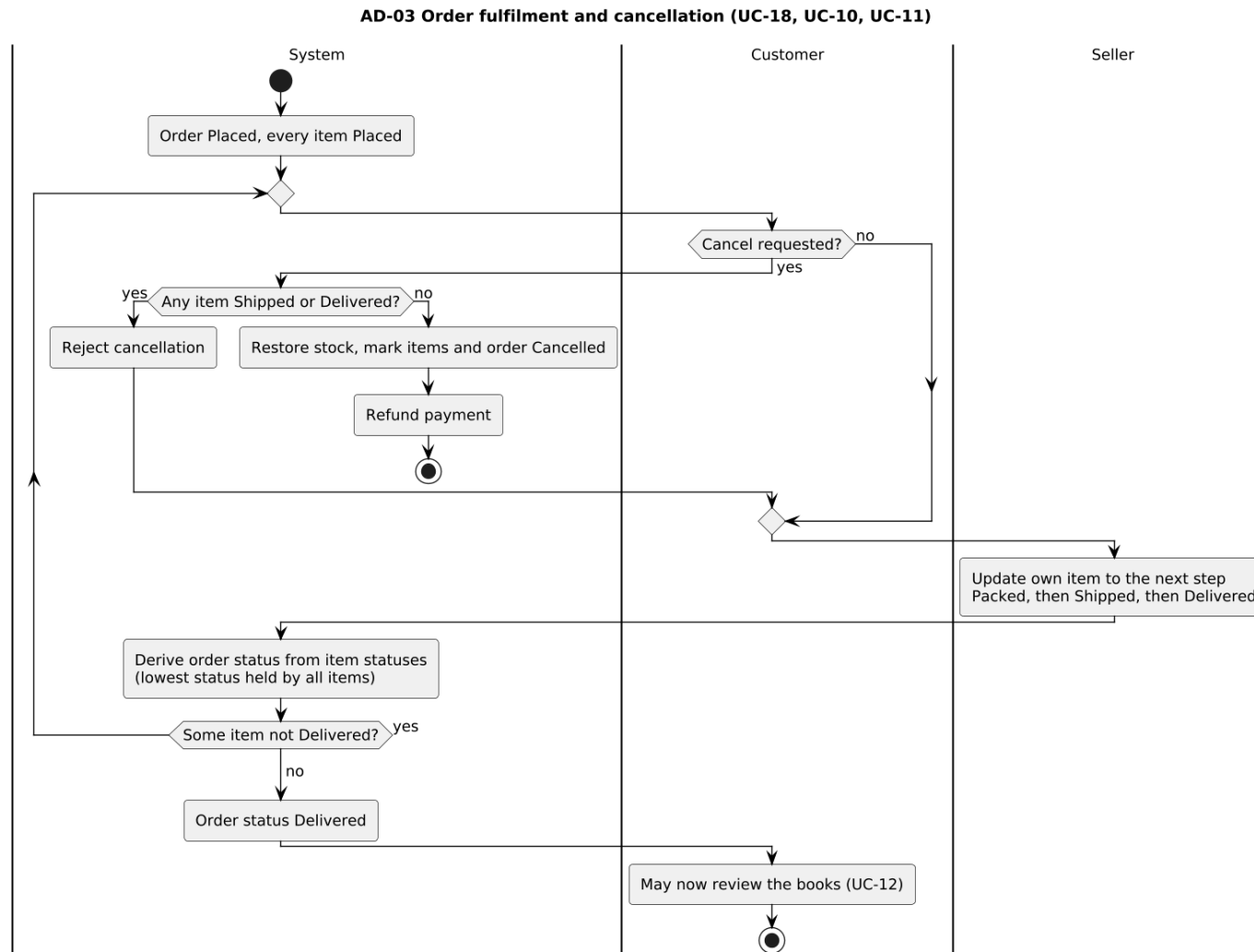
AD-01 Checkout, reservation and payment (UC-08, UC-09). Related use cases: UC-08, UC-09.

AD-02 Seller application and listing approval (UC-15, UC-16, UC-21)



AD-02 Seller application and listing approval (UC-15, UC-16, UC-21). Related use cases: UC-15, UC-16, UC-21.

AD-03 Order fulfilment and cancellation (UC-18, UC-10, UC-11)



AD-03 Order fulfilment and cancellation (UC-18, UC-10, UC-11). Related use cases: UC-18, UC-10, UC-11.

6. API Definition

6.1 Conventions

Item	Convention
Base path	/api on the same origin as the web application. The path prefix is the version; a breaking change would use /api/v2
Format	JSON request and response bodies (Content-Type: application/json), except image upload (multipart) and image download (binary)
Authentication	Session cookie set by login. Endpoints marked Public need none
CSRF	Every POST, PUT, PATCH and DELETE (except the payment webhook) requires header X-CSRF-Token with the token from GET /api/auth/csrf-token
Authorization	Role listed per endpoint; ownership rules in the services (404 for another user's resource)
Pagination	Query page (starting at 1). The page size is fixed by the server: 20 for the catalog, seller listings, administrator lists and notifications; 10 for reviews and orders. Response has items, page, pageSize, totalItems, totalPages
Money	Integers in minor currency units
Success codes	200 OK, 201 Created (new resource), 204 No Content (delete)
Error body	{ "error": { "code": "...", "message": "...", "details": [{ "field": "...", "issue": "..."}] } }
Common errors	401 UNAUTHENTICATED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 500 INTERNAL_ERROR can occur on any applicable endpoint and are not repeated in every endpoint table
Machine-readable form	openapi.yaml in this folder (OpenAPI 3.0.3)

6.2 Error code catalogue

Status	Code	Used by
400	INVALID_DATE_RANGE	API-44, API-60
400	INVALID_FILE	API-41
400	PAYMENT_VERIFICATION_FAILED	API-28
400	TOKEN_INVALID_OR_EXPIRED	API-03, API-09
400	VALIDATION_ERROR	many (36 endpoints)
401	INVALID_CREDENTIALS	API-05
401	INVALID_SIGNATURE	API-29
401	UNAUTHENTICATED	API-06, API-07, API-10, API-20, API-24
403	ACCOUNT_SUSPENDED	API-05
403	EMAIL_NOT_VERIFIED	API-05
403	NOT_ELIGIBLE	API-33
404	BOOK_NOT_FOUND	API-21, API-25
404	ITEM_NOT_IN_CART	API-22, API-23
404	NOT_FOUND	many (23 endpoints)
409	ADDRESS_LIMIT	API-12
409	ALREADY_REVIEWED	API-33
409	ALREADY_SELLER	API-34
409	APPLICATION_EXISTS	API-34
409	CANNOT_MODIFY_SELF	API-48
409	CATEGORY_EXISTS	API-55, API-56
409	CATEGORY_IN_USE	API-57
409	EMAIL_TAKEN	API-02
409	INSUFFICIENT_STOCK	API-21, API-22, API-27
409	INVALID_STATUS_TRANSITION	API-43
409	ISBN_EXISTS_FOR_SELLER	API-37

Status	Code	Used by
409	NOT_PENDING	API-50, API-51, API-53, API-54
409	ORDER_FAILED_REFUNDED	API-28
409	ORDER_NOT_CANCELLABLE	API-32
409	ORDER_NOT_PAYABLE	API-28
409	STOCK_BELOW_RESERVED	API-40
413	FILE_TOO_LARGE	API-41
423	ACCOUNT_LOCKED	API-05
429	RATE_LIMITED	API-02, API-04, API-05, API-08
502	PAYMENT_GATEWAY_ERROR	API-27, API-28
503	UNHEALTHY	API-61
401	UNAUTHENTICATED	every protected endpoint
403	FORBIDDEN	every protected endpoint
403	CSRF_INVALID	every state-changing endpoint except the webhook
500	INTERNAL_ERROR	any

6.3 Endpoint summary

The API has 61 endpoints. The list was built from the use cases and requirements, not from a template. Endpoints marked with a requirement of "(system)" support the system as a whole.

ID	Method	Path	Access	Module	Requirements	Use cases
API-01	GET	/api/auth/csrf-token	None	BM-15	VFR-06	UC-04, UC-05, UC-14
API-02	POST	/api/auth/register	None	BM-01	FR-01, VFR-04	UC-04, UC-25
API-03	POST	/api/auth/verify-email	None	BM-01	FR-01	UC-04
API-04	POST	/api/auth/resend-verification	None	BM-01	FR-01	UC-04, UC-25
API-05	POST	/api/auth/login	None	BM-01	FR-02, FR-24, VFR-07	UC-05
API-06	POST	/api/auth/logout	Customer, Seller or Administrator	BM-01	FR-02	UC-05
API-07	GET	/api/auth/me	Customer, Seller or Administrator	BM-01	FR-02, FR-29	UC-05
API-08	POST	/api/auth/forgot-password	None	BM-01	FR-03	UC-14, UC-25
API-09	POST	/api/auth/reset-password	None	BM-01	FR-03, VFR-04	UC-14
API-10	GET	/api/profile	Customer	BM-02	FR-18	UC-13
API-11	PATCH	/api/profile	Customer	BM-02	FR-18	UC-13
API-12	POST	/api/profile/addresses	Customer	BM-02	FR-18, FR-11	UC-13, UC-08
API-13	PATCH	/api/profile/addresses/:addressId	Customer	BM-02	FR-18	UC-13
API-14	DELETE	/api/profile/addresses/:addressId	Customer	BM-02	FR-18	UC-13
API-15	GET	/api/books	None	BM-03	FR-04, FR-05, FR-06, VFR-01, VFR-02	UC-01, UC-02
API-16	GET	/api/books/:bookId	None	BM-03	FR-07, VFR-01	UC-03

ID	Method	Path	Access	Module	Requirements	Use cases
API-17	GET	/api/books/:bookId/reviews	None	BM-08	FR-07, FR-17	UC-03, UC-12
API-18	GET	/api/categories	None	BM-03	FR-06, FR-26	UC-02, UC-23
API-19	GET	/api/images/:imageId	None	BM-14	FR-07, FR-20	UC-03, UC-16
API-20	GET	/api/cart	Customer	BM-04	FR-08, FR-12	UC-06, UC-08
API-21	POST	/api/cart/items	Customer	BM-04	FR-08, FR-09	UC-06
API-22	PATCH	/api/cart/items/:bookId	Customer	BM-04	FR-08, FR-09	UC-06
API-23	DELETE	/api/cart/items/:bookId	Customer	BM-04	FR-08	UC-06
API-24	GET	/api/wishlist	Customer	BM-05	FR-10	UC-07
API-25	PUT	/api/wishlist/items/:bookId	Customer	BM-05	FR-10	UC-07
API-26	DELETE	/api/wishlist/items/:bookId	Customer	BM-05	FR-10	UC-07
API-27	POST	/api/orders	Customer	BM-06	FR-11, FR-12, FR-13, FR-30, VFR-08, VFR-14	UC-08, UC-09
API-28	POST	/api/orders/:orderId/payment/confirm	Customer	BM-06	FR-13, FR-14, FR-30, VFR-08, VFR-14	UC-08, UC-09, UC-25
API-29	POST	/api/payments/webhook	External Payment Gateway	BM-07	FR-13, VFR-08	UC-09
API-30	GET	/api/orders	Customer	BM-06	FR-15	UC-10
API-31	GET	/api/orders/:orderId	Customer	BM-06	FR-15	UC-10
API-32	POST	/api/orders/:orderId/cancel	Customer	BM-06	FR-16, VFR-14	UC-11
API-33	POST	/api/books/:bookId/reviews	Customer	BM-08	FR-17	UC-12
API-34	POST	/api/seller/applications	Customer	BM-09	FR-19	UC-15
API-35	GET	/api/seller/applications/me	Customer	BM-09	FR-19	UC-15

ID	Method	Path	Access	Module	Requirements	Use cases
API-36	GET	/api/seller/books	Seller	BM-16	FR-20	UC-16
API-37	POST	/api/seller/books	Seller	BM-16	FR-20, FR-25	UC-16
API-38	PATCH	/api/seller/books/:bookId	Seller	BM-16	FR-20	UC-16
API-39	DELETE	/api/seller/books/:bookId	Seller	BM-16	FR-20	UC-16
API-40	PATCH	/api/seller/books/:bookId/inventory	Seller	BM-10	FR-21, VFR-14	UC-17
API-41	POST	/api/seller/uploads/images	Seller	BM-14	FR-20	UC-16
API-42	GET	/api/seller/orders	Seller	BM-09	FR-22	UC-18
API-43	PATCH	/api/seller/orders/:orderId/items/:itemId/status	Seller	BM-09	FR-22, FR-15	UC-18
API-44	GET	/api/seller/reports/sales	Seller	BM-12	FR-23	UC-19
API-45	GET	/api/notifications	Seller	BM-13	FR-21	UC-17, UC-25
API-46	PATCH	/api/notifications/:notificationId/read	Seller	BM-13	FR-21	UC-17
API-47	GET	/api/admin/users	Administrator	BM-11	FR-24	UC-20
API-48	PATCH	/api/admin/users/:userId/status	Administrator	BM-11	FR-24	UC-20
API-49	GET	/api/admin/seller-applications	Administrator	BM-11	FR-25	UC-21
API-50	POST	/api/admin/seller-applications/:applicationId/approve	Administrator	BM-09	FR-19, FR-25	UC-21
API-51	POST	/api/admin/seller-applications/:applicationId/reject	Administrator	BM-09	FR-25	UC-21
API-52	GET	/api/admin/books	Administrator	BM-11	FR-25	UC-21
API-53	POST	/api/admin/books/:bookId/approve	Administrator	BM-16	FR-25	UC-21
API-54	POST	/api/admin/books/:bookId/reject	Administrator	BM-16	FR-25	UC-21

ID	Method	Path	Access	Module	Requirements	Use cases
API-55	POST	/api/admin/categories	Administrator	BM-03	FR-26	UC-23
API-56	PATCH	/api/admin/categories/:categoryId	Administrator	BM-03	FR-26	UC-23
API-57	DELETE	/api/admin/categories/:categoryId	Administrator	BM-03	FR-26	UC-23
API-58	GET	/api/admin/reviews	Administrator	BM-08	FR-27	UC-22
API-59	DELETE	/api/admin/reviews/:reviewId	Administrator	BM-08	FR-27	UC-22
API-60	GET	/api/admin/reports/platform	Administrator	BM-12	FR-28	UC-24
API-61	GET	/api/health	None	BM-15	VFR-09	UC-01 to UC-25 (system-wide)

Supporting endpoints that no single FR names but that a use case needs to work: API-01 (CSRF token), API-04 (resend verification, because a link can expire), API-07 (current user for the SPA), API-35 (application status), API-45 and API-46 (in-app low-stock alerts for FR-21), API-19 (book images) and API-61 (health check for VFR-09).

6.4 Endpoint definitions

Authentication and platform

API-01 GET /api/auth/csrf-token

Issue a CSRF token

Item	Definition
Access	None
Authentication	None (public)
CSRF	Not applicable
Module	BM-15 Platform and Security
Requirements and use cases	VFR-06; UC-04, UC-05, UC-14
Success	200: { csrfToken } and sets the CSRF cookie
Errors	None specific (common errors apply)
Validation rules	None

API-02 POST /api/auth/register

Register a customer account

Item	Definition
Access	None
Authentication	None (public)
CSRF	Required (X-CSRF-Token)
Module	BM-01 Authentication
Requirements and use cases	FR-01, VFR-04; UC-04, UC-25
Request body	name (required): string (1 to 100 chars) email (required): string (email) password (required): string (8 to 72 chars)
Success	201: { message } (verification email queued)
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 409 EMAIL_TAKEN: Email already registered 429 RATE_LIMITED: Too many requests from this client
Validation rules	name 1-100 chars; email valid, stored lower-case; password 8-72 chars

API-03 POST /api/auth/verify-email

Verify email address

Item	Definition
Access	None
Authentication	None (public)
CSRF	Required (X-CSRF-Token)
Module	BM-01 Authentication
Requirements and use cases	FR-01; UC-04
Request body	token (required): string (32 to 128 chars)
Success	200: { message }; account becomes active
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 400 TOKEN_INVALID_OR_EXPIRED: Token unknown, expired or already used
Validation rules	token is a 32-128 character string

API-04 POST /api/auth/resend-verification

Resend verification email

Item	Definition
Access	None
Authentication	None (public)
CSRF	Required (X-CSRF-Token)
Module	BM-01 Authentication
Requirements and use cases	FR-01; UC-04, UC-25
Request body	email (required): string (email)
Success	200: { message }; always the same text so existing emails are not revealed
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 429 RATE_LIMITED: Too many requests from this client
Validation rules	email valid

API-05 POST /api/auth/login

Log in

Item	Definition
Access	None
Authentication	None (public)
CSRF	Required (X-CSRF-Token)
Module	BM-01 Authentication
Requirements and use cases	FR-02, FR-24, VFR-07; UC-05
Request body	email (required): string (email) password (required): string (1 to 72 chars)
Success	200: User object; session cookie set (HttpOnly, Secure, SameSite=Lax, 60 minutes)
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 401 INVALID_CREDENTIALS: Email or password wrong (same response for both) 403 EMAIL_NOT_VERIFIED: Email not yet verified (login requires verification: design assumption A-02) 403 ACCOUNT_SUSPENDED: Account suspended by an administrator 423 ACCOUNT_LOCKED: 5 consecutive failures; locked for 15 minutes; Retry-After header set 429 RATE_LIMITED: Too many requests from this client
Validation rules	email valid; password present

API-06 POST /api/auth/logout

Log out

Item	Definition
Access	Customer, Seller or Administrator
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-01 Authentication
Requirements and use cases	FR-02; UC-05
Success	200: { message }; cookie cleared; token version incremented
Errors	401 UNAUTHENTICATED: No valid session
Validation rules	None

API-07 GET /api/auth/me

Get current user

Item	Definition
Access	Customer, Seller or Administrator
Authentication	Session cookie
CSRF	Not applicable
Module	BM-01 Authentication
Requirements and use cases	FR-02, FR-29; UC-05
Success	200: Current user (id, name, email, roles, status)
Errors	401 UNAUTHENTICATED: No valid session
Validation rules	None

API-08 POST /api/auth/forgot-password

Request password reset email

Item	Definition
Access	None
Authentication	None (public)
CSRF	Required (X-CSRF-Token)
Module	BM-01 Authentication
Requirements and use cases	FR-03; UC-14, UC-25
Request body	email (required): string (email)
Success	200: { message }; always the same text
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 429 RATE_LIMITED: Too many requests from this client
Validation rules	email valid

API-09 POST /api/auth/reset-password

Reset password

Item	Definition
Access	None
Authentication	None (public)
CSRF	Required (X-CSRF-Token)
Module	BM-01 Authentication
Requirements and use cases	FR-03, VFR-04; UC-14
Request body	token (required): string (32 to 128 chars) newPassword (required): string (8 to 72 chars)
Success	200: { message }; token consumed; all sessions invalidated
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 400 TOKEN_INVALID_OR_EXPIRED: Token unknown, expired (30 minutes) or already used
Validation rules	newPassword 8-72 chars; token 32-128 chars

Profile and addresses

API-10 GET /api/profile

Get own profile and addresses

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Not applicable
Module	BM-02 User and Profile

Item	Definition
Requirements and use cases	FR-18; UC-13
Success	200: Profile with name, email, phone and addresses
Errors	401 UNAUTHENTICATED: No session
Validation rules	None

API-11 PATCH /api/profile

Update profile

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-02 User and Profile
Requirements and use cases	FR-18; UC-13
Request body	name: string (1 to 100 chars) phone: string (7 to 15 chars)
Success	200: Updated profile
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation
Validation rules	name 1-100 chars; phone 7-15 digits; email cannot be changed

API-12 POST /api/profile/addresses

Add a saved address

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-02 User and Profile
Requirements and use cases	FR-18, FR-11; UC-13, UC-08
Request body	label: string (0 to 30 chars) fullName (required): string (1 to 100 chars) line1 (required): string (1 to 150 chars) line2: string (0 to 150 chars) city (required): string (1 to 80 chars) state (required): string (1 to 80 chars) postalCode (required): string (3 to 12 chars) country (required): string (2 to 60 chars) phone (required): string (7 to 15 chars) isDefault: boolean
Success	201: Created address with id
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 409 ADDRESS_LIMIT: Already 5 saved addresses
Validation rules	fullName, line1, city, state, postalCode, country, phone required; maximum 5 addresses per user

API-13 PATCH /api/profile/addresses/:addressId

Update a saved address

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-02 User and Profile

Item	Definition
Requirements and use cases	FR-18; UC-13
Request body	label: string (0 to 30 chars) fullName: string (1 to 100 chars) line1: string (1 to 150 chars) line2: string (0 to 150 chars) city: string (1 to 80 chars) state: string (1 to 80 chars) postalCode: string (3 to 12 chars) country: string (2 to 60 chars) phone: string (7 to 15 chars) isDefault: boolean
Path parameters	addressId (24-character hex id)
Success	200: Updated address
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 NOT_FOUND: Resource does not exist or is not visible to the caller
Validation rules	Same field rules as creation, all optional

API-14 DELETE /api/profile/addresses/:addressId

Delete a saved address

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-02 User and Profile
Requirements and use cases	FR-18; UC-13
Path parameters	addressId (24-character hex id)
Success	204: No content
Errors	404 NOT_FOUND: Resource does not exist or is not visible to the caller
Validation rules	None

Catalog, categories and images

API-15 GET /api/books

Browse, search, filter and sort books

Item	Definition
Access	None
Authentication	None (public)
CSRF	Not applicable
Module	BM-03 Catalog
Requirements and use cases	FR-04, FR-05, FR-06, VFR-01, VFR-02; UC-01, UC-02
Query parameters	q: string (1 to 100 chars) category: id (24 hex) minPrice: integer (0 or more) maxPrice: integer (0 or more) minRating: integer (1 to 5) language: string (2 to 30 chars) sort: one of newest, price_asc, price_desc, rating_desc page: integer (1 or more)
Success	200: { items[20 max], page, pageSize: 20, totalItems, totalPages }; only approved listings
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation

Item	Definition
Validation rules	page >= 1; q 1-100 chars, lower-cased and accent-stripped, matched as a substring of title, author and ISBN through the indexed search fields; price bounds are non-negative integers (minor units); rating 1-5

API-16 GET /api/books/:bookId

Get book details

Item	Definition
Access	None
Authentication	None (public)
CSRF	Not applicable
Module	BM-03 Catalog
Requirements and use cases	FR-07, VFR-01; UC-03
Path parameters	bookId (24-character hex id)
Success	200: Book with title, author, ISBN, price, description, image URLs, stock status, average rating
Errors	404 NOT_FOUND: Resource does not exist or is not visible to the caller
Validation rules	bookId is a 24-character hex id

API-17 GET /api/books/:bookId/reviews

List reviews of a book

Item	Definition
Access	None
Authentication	None (public)
CSRF	Not applicable
Module	BM-08 Review
Requirements and use cases	FR-07, FR-17; UC-03, UC-12
Query parameters	page: integer (1 or more)
Path parameters	bookId (24-character hex id)
Success	200: { items, page, totalItems, totalPages } newest first, 10 per page
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 NOT_FOUND: Resource does not exist or is not visible to the caller
Validation rules	page >= 1

API-18 GET /api/categories

List categories

Item	Definition
Access	None
Authentication	None (public)
CSRF	Not applicable
Module	BM-03 Catalog
Requirements and use cases	FR-06, FR-26; UC-02, UC-23
Success	200: Array of { id, name }
Errors	None specific (common errors apply)
Validation rules	None

API-19 GET /api/images/:imageId

Get a book image

Item	Definition
Access	None
Authentication	None (public)
CSRF	Not applicable
Module	BM-14 Media
Requirements and use cases	FR-07, FR-20; UC-03, UC-16
Path parameters	imageId (24-character hex id)
Success	200: Image bytes with Content-Type and long-lived cache headers
Errors	404 NOT_FOUND: Resource does not exist or is not visible to the caller
Validation rules	imageId is a 24-character hex id

Cart

API-20 GET /api/cart

Get cart with totals

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Not applicable
Module	BM-04 Cart
Requirements and use cases	FR-08, FR-12; UC-06, UC-08
Success	200: Cart with items (current price, availability) and summary { subtotal, tax, shippingFee, total } in minor units
Errors	401 UNAUTHENTICATED: No session
Validation rules	None

API-21 POST /api/cart/items

Add a book to the cart

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-04 Cart
Requirements and use cases	FR-08, FR-09; UC-06
Request body	bookId (required): id (24 hex) quantity (required): integer (1 to 50)
Success	200: Updated cart; quantity is added to an existing line
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 BOOK_NOT_FOUND: Book not approved or removed 409 INSUFFICIENT_STOCK: Resulting quantity exceeds available stock (stock minus reserved)
Validation rules	quantity 1-50 and not above stock

API-22 PATCH /api/cart/items/:bookId

Change quantity of a cart line

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-04 Cart

Item	Definition
Requirements and use cases	FR-08, FR-09; UC-06
Request body	quantity (required): integer (1 to 50)
Path parameters	bookId (24-character hex id)
Success	200: Updated cart
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 ITEM_NOT_IN_CART: Book is not in the cart 409 INSUFFICIENT_STOCK: Quantity exceeds available stock (stock minus reserved)
Validation rules	quantity 1-50 and not above stock

API-23 DELETE /api/cart/items/:bookId

Remove a book from the cart

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-04 Cart
Requirements and use cases	FR-08; UC-06
Path parameters	bookId (24-character hex id)
Success	200: Updated cart
Errors	404 ITEM_NOT_IN_CART: Book is not in the cart
Validation rules	None

Wishlist

API-24 GET /api/wishlist

Get wishlist

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Not applicable
Module	BM-05 Wishlist
Requirements and use cases	FR-10; UC-07
Success	200: Wishlist with book summaries
Errors	401 UNAUTHENTICATED: No session
Validation rules	None

API-25 PUT /api/wishlist/items/:bookId

Add a book to the wishlist

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-05 Wishlist
Requirements and use cases	FR-10; UC-07
Path parameters	bookId (24-character hex id)
Success	200: Wishlist (idempotent add)
Errors	404 BOOK_NOT_FOUND: Book not approved or removed

Item	Definition
Validation rules	bookId valid

API-26 DELETE /api/wishlist/items/:bookId

Remove a book from the wishlist

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-05 Wishlist
Requirements and use cases	FR-10; UC-07
Path parameters	bookId (24-character hex id)
Success	204: No content
Errors	None specific (common errors apply)
Validation rules	bookId valid

Orders and payment

API-27 POST /api/orders

Reserve stock, create order and payment session from the cart

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-06 Order
Requirements and use cases	FR-11, FR-12, FR-13, FR-30, VFR-08, VFR-14; UC-08, UC-09
Request body	addressId: id (24 hex) shippingAddress: object saveAddress: boolean
Success	201: { order (status PendingPayment, reservationExpiresAt, summary), payment { gatewayOrderId, amount, currency, clientConfig } }; the stock of every line is reserved atomically before the payment session is created
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 409 INSUFFICIENT_STOCK: Available stock (stock minus reserved) of a cart line is too low; nothing is reserved for any line 502 PAYMENT_GATEWAY_ERROR: Gateway could not create the payment; the reservation is released
Validation rules	cart not empty; exactly one of addressId or shippingAddress; address fields as for saved addresses; an earlier unpaid order of the same customer is closed and its reservation released

API-28 POST /api/orders/:orderId/payment/confirm

Confirm payment and place the order

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-06 Order
Requirements and use cases	FR-13, FR-14, FR-30, VFR-08, VFR-14; UC-08, UC-09, UC-25
Request body	gatewayPaymentId (required): string (1 to 100 chars) gatewaySignature: string (0 to 256 chars)

Item	Definition
Path parameters	orderId (24-character hex id)
Success	200: Order with status Placed; the reservation is finalised (stock decremented); calling again returns the same confirmed order (idempotent)
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 NOT_FOUND: Resource does not exist or is not visible to the caller 400 PAYMENT_VERIFICATION_FAILED: Gateway does not confirm the payment; the reservation is kept until it expires or the order is cancelled 409 ORDER_NOT_PAYABLE: Order was cancelled or has already failed and cannot be paid 409 ORDER_FAILED_REFUNDED: Reservation had expired and the stock could not be reserved again; order failed and payment refunded 502 PAYMENT_GATEWAY_ERROR: Gateway unreachable
Validation rules	gatewayPaymentId present; order belongs to caller and is PendingPayment or already Placed

API-29 POST /api/payments/webhook

Gateway payment event callback

Item	Definition
Access	External Payment Gateway
Authentication	Gateway signature (no session)
CSRF	Not required (signature authenticated)
Module	BM-07 Payment
Requirements and use cases	FR-13, VFR-08; UC-09
Headers	X-Gateway-Signature (HMAC of the raw body with the shared webhook secret)
Success	200: { received: true }; handles payment succeeded and payment failed events (a failed event releases the reservation)
Errors	401 INVALID_SIGNATURE: Signature header does not match the raw body
Validation rules	Raw body signature verified with the shared webhook secret

API-30 GET /api/orders

List own orders

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Not applicable
Module	BM-06 Order
Requirements and use cases	FR-15; UC-10
Query parameters	page: integer (1 or more)
Success	200: { items, page, totalItems, totalPages } newest first, 10 per page; PendingPayment and PaymentFailed orders are excluded
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation
Validation rules	page >= 1

API-31 GET /api/orders/:orderId

Get one order

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Not applicable

Item	Definition
Module	BM-06 Order
Requirements and use cases	FR-15; UC-10
Path parameters	orderId (24-character hex id)
Success	200: Order with items, per-item fulfilment status, summary, address snapshot, payment status
Errors	404 NOT_FOUND: Resource does not exist or is not visible to the caller
Validation rules	orderId valid

API-32 POST /api/orders/:orderId/cancel

Cancel an order (release reservation, or restore stock and refund)

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-06 Order
Requirements and use cases	FR-16, VFR-14; UC-11
Path parameters	orderId (24-character hex id)
Success	200: Placed or Packed order: status Cancelled, stock restored, refundStatus. Unpaid order: reservation released, status PaymentFailed, no refund
Errors	404 NOT_FOUND: Resource does not exist or is not visible to the caller 409 ORDER_NOT_CANCELLABLE: An item is already Shipped or Delivered, or the order is already closed
Validation rules	Order is PendingPayment, Placed or Packed and no item is Shipped or Delivered

Reviews

API-33 POST /api/books/:bookId/reviews

Rate and review a book

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-08 Review
Requirements and use cases	FR-17; UC-12
Request body	rating (required): integer (1 to 5) comment: string (0 to 2000 chars)
Path parameters	bookId (24-character hex id)
Success	201: Created review; book average rating updated
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 NOT_FOUND: Resource does not exist or is not visible to the caller 403 NOT_ELIGIBLE: Customer has no delivered order containing this book 409 ALREADY_REVIEWED: One review per customer per book
Validation rules	rating integer 1-5; comment 0-2000 chars, stored as text and escaped on display

Seller

API-34 POST /api/seller/applications

Apply to become a seller

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-09 Seller
Requirements and use cases	FR-19; UC-15
Request body	storeName (required): string (2 to 100 chars) description: string (0 to 1000 chars)
Success	201: Application with status pending
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 409 APPLICATION_EXISTS: A pending application already exists 409 ALREADY_SELLER: Caller already has the seller role
Validation rules	storeName 2-100 chars; description 0-1000 chars

API-35 GET /api/seller/applications/me

Get own application status

Item	Definition
Access	Customer
Authentication	Session cookie
CSRF	Not applicable
Module	BM-09 Seller
Requirements and use cases	FR-19; UC-15
Success	200: Latest own application with status and reason
Errors	404 NOT_FOUND: No application submitted
Validation rules	None

API-36 GET /api/seller/books

List own listings

Item	Definition
Access	Seller
Authentication	Session cookie
CSRF	Not applicable
Module	BM-16 Book Management
Requirements and use cases	FR-20; UC-16
Query parameters	status: one of pending, approved, rejected, removed page: integer (1 or more)
Success	200: Own listings in every status
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation
Validation rules	status in pending, approved, rejected, removed; page >= 1

API-37 POST /api/seller/books

Create a listing

Item	Definition
Access	Seller
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-16 Book Management

Item	Definition
Requirements and use cases	FR-20, FR-25; UC-16
Request body	title (required): string (1 to 200 chars) author (required): string (1 to 150 chars) isbn (required): string (isbn) price (required): integer (1 or more) description: string (0 to 5000 chars) categoryId (required): id (24 hex) language (required): string (2 to 30 chars) stock (required): integer (0 to 100000) lowStockThreshold: integer (0 to 100000) imageIds: array of ids (max 3)
Success	201: Listing with status pending (not visible in the catalog until approved); search fields are generated by the server
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 409 ISBN_EXISTS_FOR_SELLER: Seller already has a listing with this ISBN
Validation rules	title 1-200; author 1-150; ISBN-10 or ISBN-13 with valid check digit; price positive integer (minor units); stock 0-100000; at most 3 imageIds owned by the seller

API-38 PATCH /api/seller/books/:bookId

Edit a listing

Item	Definition
Access	Seller
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-16 Book Management
Requirements and use cases	FR-20; UC-16
Request body	title: string (1 to 200 chars) author: string (1 to 150 chars) price: integer (1 or more) description: string (0 to 5000 chars) categoryId: id (24 hex) language: string (2 to 30 chars) imageIds: array of ids (max 3)
Path parameters	bookId (24-character hex id)
Success	200: Updated listing (approval status unchanged)
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 NOT_FOUND: Resource does not exist or is not visible to the caller
Validation rules	Same rules as creation, all optional; stock is changed only through the inventory endpoint

API-39 DELETE /api/seller/books/:bookId

Remove a listing

Item	Definition
Access	Seller
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-16 Book Management
Requirements and use cases	FR-20; UC-16
Path parameters	bookId (24-character hex id)
Success	204: No content; listing status set to removed (history kept)
Errors	404 NOT_FOUND: Resource does not exist or is not visible to the caller
Validation rules	None

API-40 PATCH /api/seller/books/:bookId/inventory

Update stock and low-stock threshold

Item	Definition
Access	Seller
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-10 Inventory
Requirements and use cases	FR-21, VFR-14; UC-17
Request body	stock: integer (0 to 100000) lowStockThreshold: integer (0 to 100000)
Path parameters	bookId (24-character hex id)
Success	200: Listing with new stock, available units, stockStatus and threshold
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 NOT_FOUND: Resource does not exist or is not visible to the caller 409 STOCK_BELOW_RESERVED: New stock is lower than the units reserved by open checkouts
Validation rules	stock 0-100000 and not below the units currently reserved; threshold 0-100000; at least one field

API-41 POST /api/seller/uploads/images

Upload a book image

Item	Definition
Access	Seller
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-14 Media
Requirements and use cases	FR-20; UC-16
Request body	file (required): file (multipart)
Success	201: { imageUrl, url }
Errors	400 INVALID_FILE: Not a JPEG or PNG image 413 FILE_TOO_LARGE: File larger than 2 MB
Validation rules	multipart field file; JPEG or PNG; at most 2 MB; content type checked from file signature, not from the file name

API-42 GET /api/seller/orders

List orders containing own items

Item	Definition
Access	Seller
Authentication	Session cookie
CSRF	Not applicable
Module	BM-09 Seller
Requirements and use cases	FR-22; UC-18
Query parameters	status: one of Placed, Packed, Shipped, Delivered, Cancelled page: integer (1 or more)
Success	200: Orders containing the seller's items; response contains only the seller's own items plus shipping name and address
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation
Validation rules	status filter on item status; page >= 1

API-43 PATCH /api/seller/orders/:orderId/items/:itemId/status

Update fulfilment status of an own item

Item	Definition
Access	Seller
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-09 Seller
Requirements and use cases	FR-22, FR-15; UC-18
Request body	status (required): one of Packed, Shipped, Delivered
Path parameters	orderId (24-character hex id), itemId (24-character hex id)
Success	200: Order with updated item status and re-derived order status
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 NOT_FOUND: Resource does not exist or is not visible to the caller 409 INVALID_STATUS_TRANSITION: Only Placed to Packed to Shipped to Delivered, one step at a time
Validation rules	status is the next step after the current item status

API-44 GET /api/seller/reports/sales

Sales report for a date range

Item	Definition
Access	Seller
Authentication	Session cookie
CSRF	Not applicable
Module	BM-12 Reporting
Requirements and use cases	FR-23; UC-19
Query parameters	from (required): date (YYYY-MM-DD) to (required): date (YYYY-MM-DD)
Success	200: { from, to, orderCount, unitsSold, revenue, byBook[] } for the seller only
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 400 INVALID_DATE_RANGE: from after to, or range longer than 366 days
Validation rules	from and to are ISO dates; from <= to; range <= 366 days

Notifications

API-45 GET /api/notifications

List own in-app alerts

Item	Definition
Access	Seller
Authentication	Session cookie
CSRF	Not applicable
Module	BM-13 Notification
Requirements and use cases	FR-21; UC-17, UC-25
Query parameters	unread: boolean page: integer (1 or more)
Success	200: In-app alerts (low stock) newest first, 20 per page
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation
Validation rules	page >= 1

API-46 PATCH /api/notifications/:notificationId/read

Mark an alert as read

Item	Definition
Access	Seller
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-13 Notification
Requirements and use cases	FR-21; UC-17
Path parameters	notificationId (24-character hex id)
Success	200: Alert with readAt set
Errors	404 NOT_FOUND: Resource does not exist or is not visible to the caller
Validation rules	notificationId valid

Administrator

API-47 GET /api/admin/users

List users

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Not applicable
Module	BM-11 Admin
Requirements and use cases	FR-24; UC-20
Query parameters	role: one of customer, seller, admin status: one of pending_verification, active, suspended q: string (1 to 100 chars) page: integer (1 or more)
Success	200: { items (no password data), page, totalItems, totalPages }
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation
Validation rules	page >= 1; role and status from the allowed sets

API-48 PATCH /api/admin/users/:userId/status

Suspend or reactivate a user

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-11 Admin
Requirements and use cases	FR-24; UC-20
Request body	status (required): one of active, suspended
Path parameters	userId (24-character hex id)
Success	200: User with new status; sessions of a suspended user invalidated
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 NOT_FOUND: Resource does not exist or is not visible to the caller 409 CANNOT_MODIFY_SELF: Administrators cannot suspend their own account
Validation rules	status is active or suspended

API-49 GET /api/admin/seller-applications*List seller applications*

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Not applicable
Module	BM-11 Admin
Requirements and use cases	FR-25; UC-21
Query parameters	status: one of pending, approved, rejected page: integer (1 or more)
Success	200: Seller applications, default status pending
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation
Validation rules	status in allowed set; page >= 1

API-50 POST /api/admin/seller-applications/:applicationId/approve*Approve a seller application*

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-09 Seller
Requirements and use cases	FR-19, FR-25; UC-21
Path parameters	applicationId (24-character hex id)
Success	200: Application approved; applicant gains the seller role
Errors	404 NOT_FOUND: Resource does not exist or is not visible to the caller 409 NOT_PENDING: Application already decided
Validation rules	application must be pending

API-51 POST /api/admin/seller-applications/:applicationId/reject*Reject a seller application*

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-09 Seller
Requirements and use cases	FR-25; UC-21
Request body	reason (required): string (1 to 500 chars)
Path parameters	applicationId (24-character hex id)
Success	200: Application rejected with reason
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 NOT_FOUND: Resource does not exist or is not visible to the caller 409 NOT_PENDING: Application already decided
Validation rules	reason 1-500 chars

API-52 GET /api/admin/books*List listings for approval*

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Not applicable
Module	BM-11 Admin
Requirements and use cases	FR-25; UC-21
Query parameters	status: one of pending, approved, rejected, removed page: integer (1 or more)
Success	200: Listings by status, default pending
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation
Validation rules	status in allowed set; page >= 1

API-53 POST /api/admin/books/:bookId/approve

Approve a listing

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-16 Book Management
Requirements and use cases	FR-25; UC-21
Path parameters	bookId (24-character hex id)
Success	200: Listing approved and visible in the catalog
Errors	404 NOT_FOUND: Resource does not exist or is not visible to the caller 409 NOT_PENDING: Listing already decided
Validation rules	listing must be pending

API-54 POST /api/admin/books/:bookId/reject

Reject a listing

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-16 Book Management
Requirements and use cases	FR-25; UC-21
Request body	reason (required): string (1 to 500 chars)
Path parameters	bookId (24-character hex id)
Success	200: Listing rejected with reason
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 NOT_FOUND: Resource does not exist or is not visible to the caller 409 NOT_PENDING: Listing already decided
Validation rules	reason 1-500 chars

API-55 POST /api/admin/categories

Add a category

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)

Item	Definition
Module	BM-03 Catalog
Requirements and use cases	FR-26; UC-23
Request body	name (required): string (1 to 60 chars)
Success	201: Created category
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 409 CATEGORY_EXISTS: Name already used (case-insensitive)
Validation rules	name 1-60 chars, unique ignoring case

API-56 PATCH /api/admin/categories/:categoryId

Rename a category

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-03 Catalog
Requirements and use cases	FR-26; UC-23
Request body	name (required): string (1 to 60 chars)
Path parameters	categoryId (24-character hex id)
Success	200: Renamed category
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 404 NOT_FOUND: Resource does not exist or is not visible to the caller 409 CATEGORY_EXISTS: Name already used (case-insensitive)
Validation rules	name 1-60 chars, unique ignoring case

API-57 DELETE /api/admin/categories/:categoryId

Delete a category

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-03 Catalog
Requirements and use cases	FR-26; UC-23
Path parameters	categoryId (24-character hex id)
Success	204: No content
Errors	404 NOT_FOUND: Resource does not exist or is not visible to the caller 409 CATEGORY_IN_USE: Books still reference the category
Validation rules	category must have no books

API-58 GET /api/admin/reviews

List reviews for moderation

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Not applicable
Module	BM-08 Review
Requirements and use cases	FR-27; UC-22

Item	Definition
Query parameters	bookId: id (24 hex) maxRating: integer (1 to 5) page: integer (1 or more)
Success	200: All reviews for moderation, newest first
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation
Validation rules	page >= 1; bookId valid id

API-59 DELETE /api/admin/reviews/:reviewId

Remove a review

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Required (X-CSRF-Token)
Module	BM-08 Review
Requirements and use cases	FR-27; UC-22
Path parameters	reviewId (24-character hex id)
Success	204: No content; book average rating recalculated
Errors	404 NOT_FOUND: Resource does not exist or is not visible to the caller
Validation rules	reviewId valid

API-60 GET /api/admin/reports/platform

Platform report for a date range

Item	Definition
Access	Administrator
Authentication	Session cookie
CSRF	Not applicable
Module	BM-12 Reporting
Requirements and use cases	FR-28; UC-24
Query parameters	from (required): date (YYYY-MM-DD) to (required): date (YYYY-MM-DD)
Success	200: { from, to, orderCount, revenue, topBooks[10] }
Errors	400 VALIDATION_ERROR: Request body, query or path parameter fails validation 400 INVALID_DATE_RANGE: from after to, or range longer than 366 days
Validation rules	from and to are ISO dates; from <= to; range <= 366 days

System

API-61 GET /api/health

Health check used by the uptime monitor

Item	Definition
Access	None
Authentication	None (public)
CSRF	Not applicable
Module	BM-15 Platform and Security
Requirements and use cases	VFR-09; UC-01 to UC-25 (system-wide)
Success	200: { status: ok, db: up, uptimeSeconds, time }; 503 with db: down when the database does not answer
Errors	503 UNHEALTHY: Database check failed

Item	Definition
Validation rules	None

6.5 OpenAPI specification (VFR-15)

`openapi.yaml` describes all 61 endpoints with parameters, request bodies, responses, security schemes and shared schemas. It is generated from the same endpoint table as Sections 6.3 and 6.4, so the three always agree, and it is validated with an OpenAPI validator. VFR-15 is verified by checking that every endpoint in Section 6.3 appears in the file and that the file validates (Validation Specification, TC-VFR-15).

6.6 Role-based access matrix (FR-29)

The columns are roles, not kinds of people. A seller account has both the customer role and the seller role (D-04; there is no Customer-versus-Seller exclusion), so it can use every endpoint marked for the Customer role as well as the Seller endpoints. An administrator account has only the administrator role. Y means a caller who holds that role may use the endpoint.

ID	Method	Path	Guest	Customer role	Seller role	Administrator role	Extra rule
API-01	GET	/api/auth/csrf-token	Y	Y	Y	Y	None
API-02	POST	/api/auth/register	Y	Y	Y	Y	None
API-03	POST	/api/auth/verify-email	Y	Y	Y	Y	None
API-04	POST	/api/auth/resend-verification	Y	Y	Y	Y	None
API-05	POST	/api/auth/login	Y	Y	Y	Y	None
API-06	POST	/api/auth/logout	-	Y	Y	Y	Any logged-in user
API-07	GET	/api/auth/me	-	Y	Y	Y	Any logged-in user
API-08	POST	/api/auth/forgot-password	Y	Y	Y	Y	None
API-09	POST	/api/auth/reset-password	Y	Y	Y	Y	None
API-10	GET	/api/profile	-	Y	-	-	Role only
API-11	PATCH	/api/profile	-	Y	-	-	Role only
API-12	POST	/api/profile/addresses	-	Y	-	-	Role only
API-13	PATCH	/api/profile/addresses/:addressId	-	Y	-	-	Owner only (404 for others)
API-14	DELETE	/api/profile/addresses/:addressId	-	Y	-	-	Owner only (404 for others)
API-15	GET	/api/books	Y	Y	Y	Y	None
API-16	GET	/api/books/:bookId	Y	Y	Y	Y	None
API-17	GET	/api/books/:bookId/reviews	Y	Y	Y	Y	None
API-18	GET	/api/categories	Y	Y	Y	Y	None
API-19	GET	/api/images/:imageId	Y	Y	Y	Y	None
API-20	GET	/api/cart	-	Y	-	-	Role only
API-21	POST	/api/cart/items	-	Y	-	-	Role only
API-22	PATCH	/api/cart/items/:bookId	-	Y	-	-	Role only
API-23	DELETE	/api/cart/items/:bookId	-	Y	-	-	Role only
API-24	GET	/api/wishlist	-	Y	-	-	Role only
API-25	PUT	/api/wishlist/items/:bookId	-	Y	-	-	Role only
API-26	DELETE	/api/wishlist/items/:bookId	-	Y	-	-	Role only

ID	Method	Path	Guest	Customer role	Seller role	Administrator role	Extra rule
API-27	POST	/api/orders	-	Y	-	-	Role only
API-28	POST	/api/orders/:orderId/payment/confirm	-	Y	-	-	Owner only (404 for others)
API-29	POST	/api/payments/webhook	-	-	-	-	Signature only
API-30	GET	/api/orders	-	Y	-	-	Role only
API-31	GET	/api/orders/:orderId	-	Y	-	-	Owner only (404 for others)
API-32	POST	/api/orders/:orderId/cancel	-	Y	-	-	Owner only (404 for others)
API-33	POST	/api/books/:bookId/reviews	-	Y	-	-	Role only
API-34	POST	/api/seller/applications	-	Y	-	-	Role only
API-35	GET	/api/seller/applications/me	-	Y	-	-	Role only
API-36	GET	/api/seller/books	-	-	Y	-	Owner only (404 for others)
API-37	POST	/api/seller/books	-	-	Y	-	Role only
API-38	PATCH	/api/seller/books/:bookId	-	-	Y	-	Owner only (404 for others)
API-39	DELETE	/api/seller/books/:bookId	-	-	Y	-	Owner only (404 for others)
API-40	PATCH	/api/seller/books/:bookId/inventory	-	-	Y	-	Owner only (404 for others)
API-41	POST	/api/seller/uploads/images	-	-	Y	-	Role only
API-42	GET	/api/seller/orders	-	-	Y	-	Owner only (404 for others)
API-43	PATCH	/api/seller/orders/:orderId/items/:itemId/status	-	-	Y	-	Owner only (404 for others)
API-44	GET	/api/seller/reports/sales	-	-	Y	-	Owner only (404 for others)
API-45	GET	/api/notifications	-	-	Y	-	Owner only (404 for others)
API-46	PATCH	/api/notifications/:notificationId/read	-	-	Y	-	Owner only (404 for others)
API-47	GET	/api/admin/users	-	-	-	Y	Role only
API-48	PATCH	/api/admin/users/:userId/status	-	-	-	Y	Role only
API-49	GET	/api/admin/seller-applications	-	-	-	Y	Role only
API-50	POST	/api/admin/seller-applications/:applicationId/approve	-	-	-	Y	Role only
API-51	POST	/api/admin/seller-applications/:applicationId/reject	-	-	-	Y	Role only
API-52	GET	/api/admin/books	-	-	-	Y	Role only
API-53	POST	/api/admin/books/:bookId/approve	-	-	-	Y	Role only
API-54	POST	/api/admin/books/:bookId/reject	-	-	-	Y	Role only
API-55	POST	/api/admin/categories	-	-	-	Y	Role only
API-56	PATCH	/api/admin/categories/:categoryId	-	-	-	Y	Role only
API-57	DELETE	/api/admin/categories/:categoryId	-	-	-	Y	Role only
API-58	GET	/api/admin/reviews	-	-	-	Y	Role only

ID	Method	Path	Guest	Customer role	Seller role	Administrator role	Extra rule
API-59	DELETE	/api/admin/reviews/:reviewId	-	-	-	Y	Role only
API-60	GET	/api/admin/reports/platform	-	-	-	Y	Role only
API-61	GET	/api/health	Y	Y	Y	Y	None

7. ABI and Internal Interface Definition

7.1 What "ABI" means in this project

An Application Binary Interface (ABI) normally means the binary-level contract between compiled programs: calling conventions, data layout, symbol names. The Online Bookstore has no native compiled components that call each other, so there is no binary ABI in that sense. The course brief asks for API and ABI definition and for stability of both, so this project defines its ABI as **the set of runtime contracts that must stay stable between separately built, deployed or developed parts, even though they are not source-compatible**:

1. the wire contract between the React build and the REST API (message shapes, headers, cookies, status codes);
2. the call contract between backend modules (service function signatures, results and thrown errors);
3. the persistence contract between code and the database (collection schemas and indexes);
4. the adapter contracts to the Payment Gateway and the Email Service;
5. the configuration contract (environment variable names).

"API" in this document means the public REST API (Section 6). "ABI" means the five contracts above, of which the wire contract is the one that crosses a deployment boundary. The word ABI is used for these contracts throughout Phase 2.

7.2 Contracts

Contract	Definition	Stability rule
Wire contract (frontend and REST API)	Section 6 and <code>openapi.yaml</code> : paths, methods, JSON fields, status codes, error object, cookie name <code>session</code> , header <code>X-CSRF-Token</code> , ISO dates, integer money, 24-hex ids	Within version <code>/api</code> , fields and codes are only added, never renamed, removed or given a new meaning. The frontend must ignore unknown response fields. A breaking change needs <code>/api/v2</code>
Service call contract (backend modules)	Function names and parameters in Section 4.1 and interfaces I-04 to I-24. Inputs are plain validated objects; results are plain objects; failures are <code>AppError</code> with a code from Section 6.2. Functions that take a <code>session</code> parameter run inside the caller's transaction	A function signature changes only together with all its callers in the same change. New optional parameters go last. Error codes are not reused for another meaning
Persistence contract	Collections, fields, types and indexes in Section 8. Mongoose schemas are the single definition	A field is added as optional or with a default; removal is done in two steps (stop using, then drop). Index and unique-key changes need a migration script. Stored amounts stay integers in minor units
Payment adapter contract	<code>PaymentGateway.createPayment(amount, currency, reference)</code> returns <code>{ gatewayOrderId, clientConfig }</code> ; <code>verifyPayment(reference, paymentId, signature)</code> returns boolean; <code>verifyWebhook(rawBody, signature)</code> returns the parsed event or throws; <code>refund(paymentId, amount)</code> returns <code>{ gatewayRefundId, status }</code> . No method accepts or returns card data	A new provider implements the same four methods. The fake gateway used in tests implements them with the same signatures
Email adapter contract	<code>MailSender.send({ to, subject, text, html })</code> returns <code>{ providerMessageId }</code> or throws <code>MailError</code>	A new provider implements the single method
Configuration contract	Environment variable names listed in the Architecture document, Section 7.2; a missing required variable stops the service at start-up	Names are not changed once deployed; new variables get a default or are optional

7.3 Who checks the contracts

Contract	How it is checked (details in the Validation Specification)
Wire	OpenAPI validation of <code>openapi.yaml</code> ; API tests compare every response with its schema; frontend ignores unknown fields

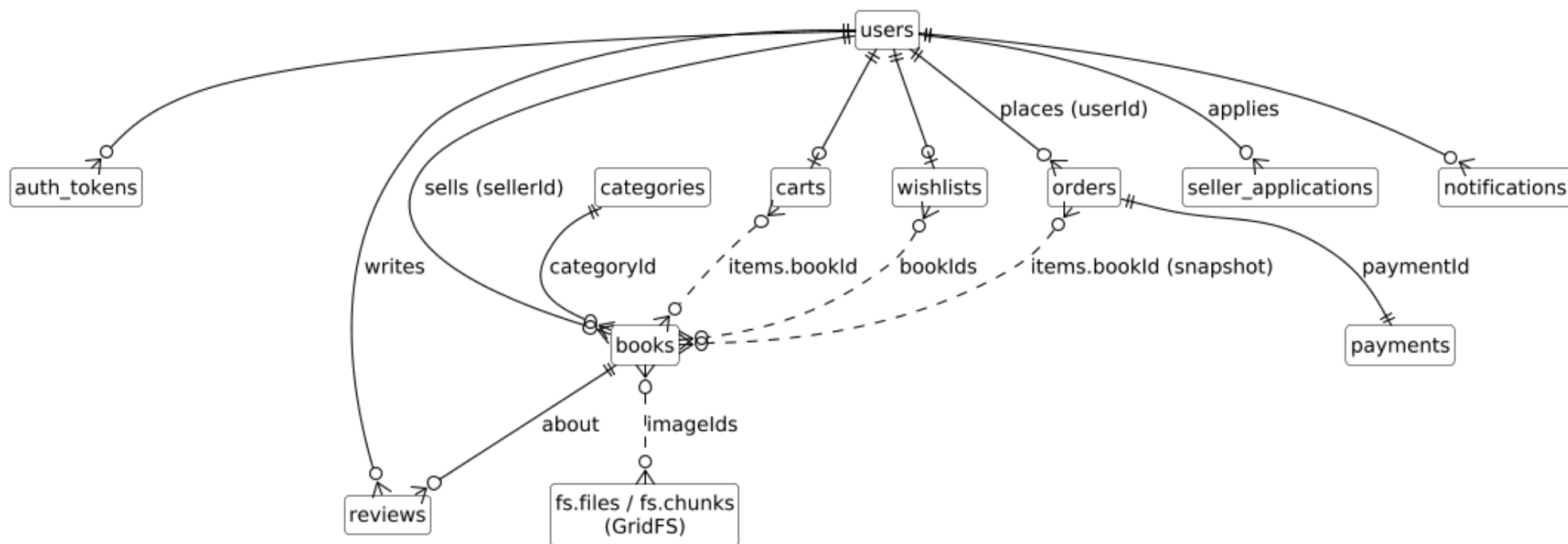
Contract	How it is checked (details in the Validation Specification)
Service call	Unit tests of services with fake repositories; integration tests with a real test database
Persistence	Schema validators and unique indexes tested for rejection of bad data
Adapters	The same contract tests run against the fake and, in the demo, against the sandbox provider
Configuration	Start-up check test: missing variable gives a clear failure

8. Database Design

8.1 ER diagrams

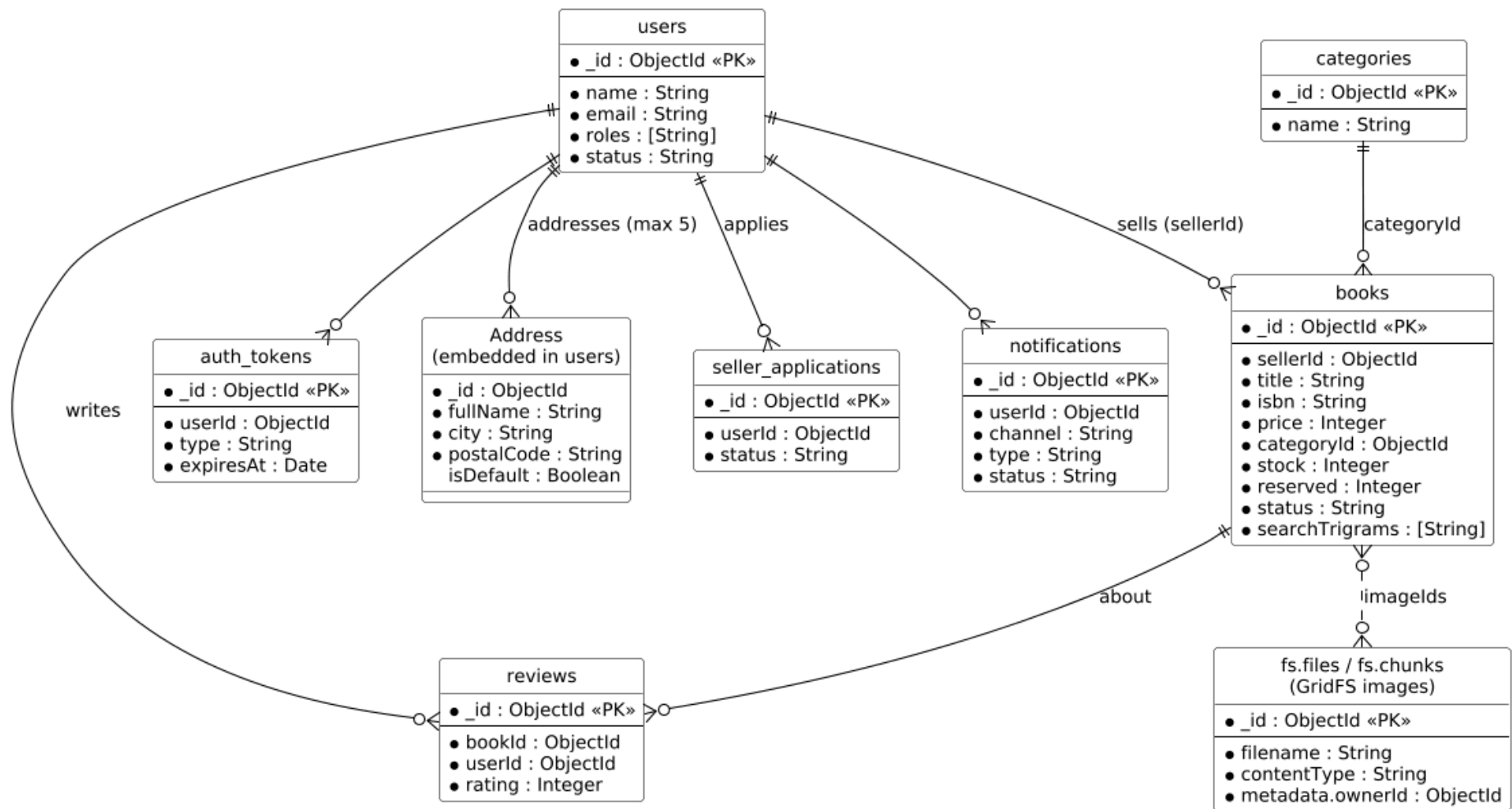
The overview shows every collection and relationship. The two detail diagrams add the key attributes: the first covers accounts, catalog and content, the second covers cart, wishlist, orders and payment. Boxes labelled "embedded" are stored inside their parent document. The complete field lists are in Section 8.2.

Database ER Overview (all collections and relationships)



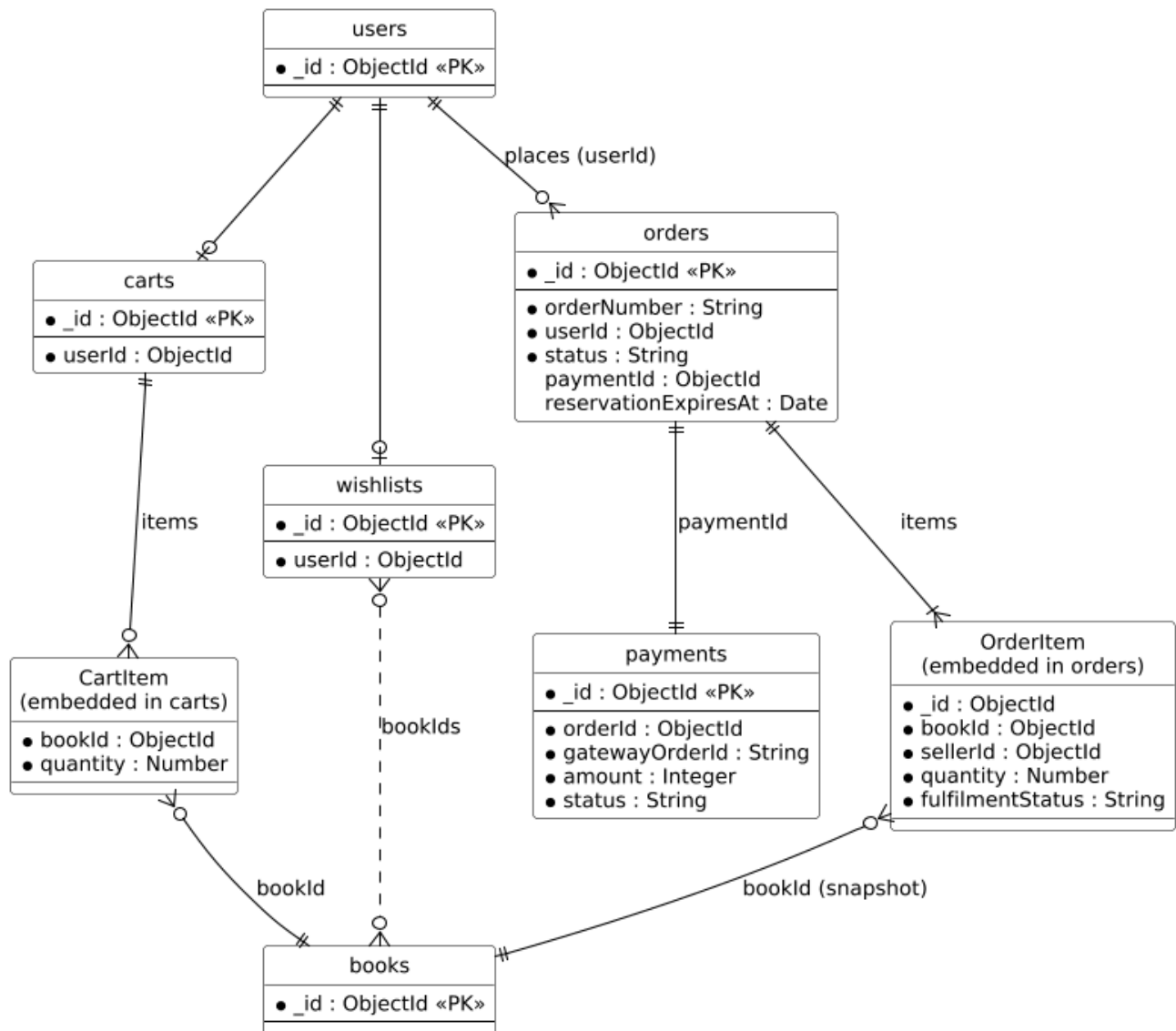
Database overview: all collections and relationships.

Database ER 1 of 2: Accounts, Catalog and Content



ER diagram 1 of 2: accounts, catalog and content, with key attributes.

Database ER 2 of 2: Cart, Wishlist, Orders and Payment



ER diagram 2 of 2: cart, wishlist, orders and payment, with key attributes.

Collection	Entity	Purpose
users	User	Every account (Customer, Seller, Administrator). Roles are a list, so one person can be both Customer and Seller.
auth_tokens	AuthToken	One-time tokens for email verification and password reset. Only a hash of the token is stored.
categories	Category	Book categories managed by administrators.
books	Book	Seller listings. Stock and reservations live on the book document so that one conditional update changes them atomically.
carts	Cart	One persistent cart per customer. Prices are not stored; they are read from books when the cart is shown.
wishlists	Wishlist	One wishlist per customer.
orders	Order	Orders with embedded items. Titles, ISBNs, prices and the address are copied at order time (snapshots) so history never changes.
payments	Payment	Payment records. Contains gateway references only; no card data of any kind.
reviews	Review	Ratings and reviews. One per customer per book.
seller_applications	SellerApplication	Requests to become a seller and the administrator decision.
notifications	Notification	Record of every email sent and the in-app low-stock alerts. Secret tokens are never stored here.

8.2 Data dictionary

Required means the field must be present. All ObjectId references are stored by value (no database-level foreign keys in MongoDB), and the services check them.

users (User)

Every account (Customer, Seller, Administrator). Roles are a list, so one person can be both Customer and Seller.

Field	Type	Required	Notes
_id	ObjectId	Y	Primary key
name	String	Y	1-100 chars
email	String	Y	Lower-case, unique
passwordHash	String	Y	bcrypt hash, cost 12 (never returned by the API)
roles	[String]	Y	Subset of customer, seller, admin; default [customer]
status	String	Y	pending_verification, active, suspended
emailVerifiedAt	Date	N	Set on verification
phone	String	N	7-15 digits
addresses	[Address]	N	Embedded, maximum 5: _id, label, fullName, line1, line2, city, state, postalCode, country, phone, isDefault
sellerProfile	Object	N	Present for sellers: storeName, description, approvedAt
failedLoginCount	Number	Y	Default 0; reset on success
lockUntil	Date	N	Set for 15 minutes after the 5th consecutive failure
tokenVersion	Number	Y	Default 0; incremented on logout, password reset, suspension
createdAt / updatedAt	Date	Y	Timestamps

Indexes: { email: 1 } unique; { roles: 1, status: 1 }

Constraints: Unique email; addresses array limited to 5 using a conditional \$push; status and roles validated by schema enum.

auth_tokens (AuthToken)

One-time tokens for email verification and password reset. Only a hash of the token is stored.

Field	Type	Required	Notes
_id	ObjectId	Y	
userId	ObjectId	Y	Reference to users
type	String	Y	email_verification or password_reset
tokenHash	String	Y	SHA-256 of the random token
expiresAt	Date	Y	30 minutes for password_reset; 24 hours for email_verification
usedAt	Date	N	Set when consumed (single use)
createdAt	Date	Y	

Indexes: { tokenHash: 1 } unique; { userId: 1, type: 1 }; { expiresAt: 1 } TTL (expireAfterSeconds: 0)

Constraints: Token accepted only when usedAt is null and expiresAt is in the future.

categories (Category)

Book categories managed by administrators.

Field	Type	Required	Notes
_id	ObjectId	Y	

Field	Type	Required	Notes
name	String	Y	1-60 chars, unique ignoring case
createdAt	Date	Y	

Indexes: { name: 1 } unique, collation strength 2 (case-insensitive)

Constraints: Cannot be deleted while a book references it.

books (Book)

Seller listings. Stock and reservations live on the book document so that one conditional update changes them atomically.

Field	Type	Required	Notes
_id	ObjectId	Y	
sellerId	ObjectId	Y	Reference to users (owner)
title	String	Y	1-200
author	String	Y	1-150
isbn	String	Y	ISBN-10 or ISBN-13, check digit validated
price	Number (integer)	Y	Minor currency units, greater than 0
description	String	N	0-5000
categoryId	ObjectId	Y	Reference to categories
language	String	Y	
imageIds	[ObjectId]	N	GridFS file ids, maximum 3; first is the cover
stock	Number (integer)	Y	Units on hand that are not yet sold; 0 or more
reserved	Number (integer)	Y	Units held by open checkouts; default 0; available = stock - reserved
lowStockThreshold	Number (integer)	Y	Default 5
status	String	Y	pending, approved, rejected, removed
rejectionReason	String	N	
avgRating	Number	Y	Default 0; maintained by Review module
reviewCount	Number	Y	Default 0
titleLower	String	Y	Derived: title lower-cased with accents removed
authorLower	String	Y	Derived: author lower-cased with accents removed
searchText	String	Y	Derived: normalised title, author and ISBN digits joined by spaces
searchTrigrams	[String]	Y	Derived: the distinct 3-character substrings of searchText; multikey indexed
createdAt / updatedAt	Date	Y	

Indexes: { sellerId: 1, isbn: 1 } unique; { status: 1, categoryId: 1, price: 1 }; { status: 1, price: 1 }; { status: 1, createdAt: -1 }; { status: 1, avgRating: -1 }; { status: 1, searchTrigrams: 1 }; { status: 1, titleLower: 1 }; { status: 1, authorLower: 1 }; { sellerId: 1, status: 1 }

Constraints: Schema validator: stock >= 0, reserved >= 0, reserved <= stock, price > 0. Reservation uses the filter { _id, status: approved, stock - reserved >= qty }. Finalisation and release are conditional on reserved >= qty. Derived: available = stock - reserved; stockStatus is "In stock" when available > 0, otherwise "Out of stock". Search fields are recomputed whenever title, author or ISBN changes.

carts (Cart)

One persistent cart per customer. Prices are not stored; they are read from books when the cart is shown.

Field	Type	Required	Notes
_id	ObjectId	Y	
userId	ObjectId	Y	Reference to users

Field	Type	Required	Notes
items	[CartItem]	Y	Embedded: bookId, quantity (1-50)
updatedAt	Date	Y	

Indexes: { userId: 1 } unique

Constraints: One cart per user; quantity limited by stock at write time.

wishlists (Wishlist)

One wishlist per customer.

Field	Type	Required	Notes
_id	ObjectId	Y	
userId	ObjectId	Y	Reference to users
bookIds	[ObjectId]	Y	Unique entries via \$addToSet
updatedAt	Date	Y	

Indexes: { userId: 1 } unique

Constraints: No duplicates.

orders (Order)

Orders with embedded items. Titles, ISBNs, prices and the address are copied at order time (snapshots) so history never changes.

Field	Type	Required	Notes
_id	ObjectId	Y	
orderNumber	String	Y	Human-readable, unique
userId	ObjectId	Y	Customer
items	[OrderItem]	Y	Embedded: _id, bookId, sellerId, titleSnapshot, isbnSnapshot, unitPrice, quantity, lineTotal, fulfilmentStatus (Placed, Packed, Shipped, Delivered, Cancelled)
shippingAddress	Object	Y	Snapshot of the chosen address
subtotal / tax / shippingFee / total	Number (integer)	Y	Minor units; total = subtotal + tax + shippingFee
status	String	Y	PendingPayment, PaymentFailed, Placed, Packed, Shipped, Delivered, Cancelled (derived from item statuses after confirmation)
paymentId	ObjectId	N	Reference to payments
reservationExpiresAt	Date	N	While PendingPayment: creation time + RESERVATION_MINUTES (default 15)
failureReason	String	N	payment_failed, abandoned, expired or stock_unavailable_refunded
placedAt	Date	N	Set at confirmation
cancelledAt	Date	N	
createdAt / updatedAt	Date	Y	

Indexes: { orderNumber: 1 } unique; { userId: 1, placedAt: -1 }; { "items.sellerId": 1, placedAt: -1 }; { status: 1, placedAt: -1 }; { status: 1, reservationExpiresAt: 1 }

Constraints: Order status is derived from item statuses (rule in the Software Design Document, Section 4.3). Creation reserves stock and confirmation finalises it, each in one transaction. Orders left in PendingPayment after reservationExpiresAt are closed by the sweeper.

payments (Payment)

Payment records. Contains gateway references only; no card data of any kind.

Field	Type	Required	Notes
_id	ObjectId	Y	
orderId	ObjectId	Y	Reference to orders
gateway	String	Y	Gateway provider name (provider TBD)
gatewayOrderId	String	Y	Reference created at the gateway
gatewayPaymentId	String	N	Set when paid
amount	Number (integer)	Y	Minor units
currency	String	Y	From configuration
status	String	Y	created, succeeded, failed, refund_pending, refunded
refund	Object	N	gatewayRefundId, amount, requestedAt, completedAt
createdAt / updatedAt	Date	Y	

Indexes: { orderId: 1 } unique; { gatewayOrderId: 1 } unique; { gatewayPaymentId: 1 } unique, sparse

Constraints: Unique gatewayPaymentId makes confirmation idempotent. No field may hold card number, expiry or security code (VFR-08).

reviews (Review)

Ratings and reviews. One per customer per book.

Field	Type	Required	Notes
_id	ObjectId	Y	
bookId	ObjectId	Y	Reference to books
userId	ObjectId	Y	Reference to users
rating	Number (integer)	Y	1-5
comment	String	N	0-2000, stored as text
createdAt	Date	Y	

Indexes: { bookId: 1, userId: 1 } unique; { bookId: 1, createdAt: -1 }

Constraints: Eligibility checked in the service against orders (a Delivered item for this user and book).

seller_applications (SellerApplication)

Requests to become a seller and the administrator decision.

Field	Type	Required	Notes
_id	ObjectId	Y	
userId	ObjectId	Y	Applicant
storeName	String	Y	2-100
description	String	N	0-1000
status	String	Y	pending, approved, rejected
rejectionReason	String	N	
decidedBy	ObjectId	N	Administrator
decidedAt	Date	N	
createdAt	Date	Y	

Indexes: { userId: 1 } unique, partial (status = "pending"); { status: 1, createdAt: 1 }

Constraints: At most one pending application per user.

notifications (Notification)

Record of every email sent and the in-app low-stock alerts. Secret tokens are never stored here.

Field	Type	Required	Notes
_id	ObjectId	Y	
userId	ObjectId	Y	Recipient
channel	String	Y	email or in_app
type	String	Y	email_verification, password_reset, order_confirmation, low_stock
subject	String	Y	
payload	Object	N	Ids only, for example { orderId } or { bookId }
status	String	Y	queued, sent, failed (email); in_app is stored as sent
attempts	Number	Y	Default 0
lastError	String	N	
sentAt	Date	N	
readAt	Date	N	In-app only
createdAt	Date	Y	

Indexes: { userId: 1, channel: 1, readAt: 1, createdAt: -1 }; { status: 1, createdAt: 1 }

Constraints: Email body is generated at send time; reset and verification links are not persisted.

8.3 How the design supports the requirements

Need	Design
No overselling (FR-30, VFR-14)	<code>stock</code> and <code>reserved</code> are fields of the <code>books</code> document. Each change (reserve, finalise, release, restore) is a conditional update on that one document, for example reserve only if <code>stock - reserved >= quantity</code> , so the check and the change are a single atomic operation. Several items are changed in one multi-document transaction, so a failure on any item undoes the others. A schema validator enforces <code>stock >= 0</code> and <code>reserved <= stock</code>
Order consistency (FR-13, VFR-14)	The order is created together with its reservation in one transaction. Confirmation (finalising the reservation, marking the order Placed, clearing the cart) is one transaction. Orders contain snapshots of title, ISBN, price and address, so later edits never change history. A unique index on <code>payments.gatewayPaymentId</code> makes confirmation idempotent. Orders left in PendingPayment after <code>reservationExpiresAt</code> are found through the index { <code>status</code> , <code>reservationExpiresAt</code> } and released by the sweeper
Seller ownership (FR-20, FR-22)	<code>books.sellerId</code> and <code>orders.items.sellerId</code> are set by the server from the session, never from the request. Every seller query includes the seller id in its filter, so another seller's data is never loaded
Review eligibility (FR-17)	A delivered item for the user and book must exist in <code>orders</code> ; the unique index { <code>bookId</code> , <code>userId</code> } allows one review per customer per book
Role-based access (FR-29)	<code>users.roles</code> is read from the database on every protected request, so a role granted by approval (UC-21) or a suspension (UC-20) takes effect immediately
Address limit (FR-18)	The address list is an embedded array; a conditional update adds an address only if the array has fewer than 5 entries, so concurrent requests cannot exceed 5
One pending seller application	A partial unique index on <code>seller_applications.userId</code> where status is pending
Search and filtering speed (VFR-02, FR-05)	Derived search fields and a multikey trigram index, plus compound indexes starting with <code>status</code> (Section 4.4 and Section 8.2)
Expiry of one-time tokens (FR-03, FR-01)	<code>auth_tokens.expiresAt</code> has a TTL index so expired tokens are removed automatically; validity is also checked in code

8.4 Transactions

Operation	Documents in the transaction
Checkout: reserve stock and create the order (FR-30)	books (reserved), orders
Confirm payment: finalise the reservation (FR-13, FR-30)	books (stock, reserved), orders, payments, carts

Operation	Documents in the transaction
Release a reservation: payment failed, unpaid order cancelled, or expiry	books (reserved), orders
Cancel a placed order (FR-16)	books (stock restored), orders
Review create or delete (FR-17, FR-27)	reviews, books (average rating)
Seller approval (FR-19, FR-25)	seller_applications, users

Calls to the Payment Gateway and the Email Service are never inside a transaction. If the gateway fails after the reservation was saved, the reservation is released in a separate step. Everything else is a single-document change and needs no transaction. MongoDB multi-document transactions need a replica set, which is why the database is deployed as one even for the course.

8.5 Initial data

Data	How it is created
Administrator account(s)	<code>seed-admin</code> script reading email and a password from the environment (Phase 1 has no use case to create administrators)
Categories	Created by the administrator (UC-23); a demo seed script creates a few for testing
Demo books, sellers and customers	<code>seed-demo-data</code> script for development and validation environments only

8.6 Image Storage with GridFS

Decision (D-09). Book images are stored in the same MongoDB database with GridFS, MongoDB's mechanism for files. No separate image service is used.

Why GridFS.

Reason	Explanation
No extra external service	Phase 1 has two external parties, the Payment Gateway and the Email Service. A third service (an object store or an image host) would need another account, keys, deployment settings and failure handling. GridFS needs none
One backup and one restore	Images are in the database, so the daily backup (VFR-14) and a restore include them. With an external store the two could get out of step
One access-control model	Images are read through the same API and owner checks as the rest of the system
Small, bounded files	At most 3 images of at most 2 MB per listing, which suits a course-sized catalogue

What is stored. Each image is a file in the `fs.files` and `fs.chunks` collections with the file name, content type, upload time and `metadata.ownerId` (the seller). A listing keeps only the list of image ids in `imageIds`; the first is the cover. Image ids are never reused, so an image never changes after it is saved.

Upload (UC-16, POST `/api/seller/uploads/images`). The seller sends one multipart file. The server rejects a file larger than 2 MB (413 `FILE_TOO_LARGE`) before reading it all, checks the first bytes of the file to confirm it is JPEG or PNG (the name and the declared type are not trusted; SVG is not allowed), then streams it into GridFS with the seller's id and returns `{ imageId, url }`. When the seller saves a listing, the server checks that every image id exists and belongs to that seller (interface I-16).

Retrieval (UC-03, GET `/api/images/:imageId`). The server streams the file from GridFS with its content type, an ETag and a long cache lifetime (public, one year), which is safe because an id never changes. Browsers and the host's cache therefore fetch each image rarely, which protects VFR-01.

Effect on backups and deployment.

Aspect	Effect
Backups	Images are included in the daily backup and in restore. Backup size and restore time grow with the number of images

Aspect	Effect
Database size	Images are the largest data in the system. As an example, 500 listings with 3 images of about 200 KB each need about 300 MB. The database tier must have enough storage (TBD, Architecture Section 11.3). The 2 MB limit is a ceiling, and the SPA should reduce large photos before upload
Deployment	No extra service, credentials or environment variables. Image delivery runs through the web service, so its bandwidth and the cache headers matter
Orphans	An uploaded image that is never attached to a listing stays in the database. A cleanup of unattached images older than a day is a possible Phase 3 task and is not required by Phase 1

Limitation. If the images grow beyond what the database tier allows, the Media module (BM-14) is the single place to change: it hides the storage behind `saveImage` and `streamImage`, so another store could be used later without changing the API. That would be a new design decision, not part of this design.

9. Division of Work

The team is Anagha N (PES1UG24CS058), Manasvi B (PES1UG24CS261), Meda Sahasra Siri (PES1UG24CS267) and Monika (PES1UG24CS274). This section is a **proposed allocation**. It states who is meant to be responsible, not that any work is complete. The team should confirm or change it. It separates **primary ownership** (who designs, builds and tests an area) from **review and support** (who checks it and who helps test it).

9.1 Primary ownership

Team member	Primary ownership (areas)
Anagha N (PES1UG24CS058)	Architecture; authentication and RBAC; integration; cross-module coordination
Manasvi B (PES1UG24CS261)	Catalog and search; book management; customer-facing catalog UI
Meda Sahasra Siri (PES1UG24CS267)	Cart; checkout and payment; orders and the customer purchase flow
Monika (PES1UG24CS274)	Seller functionality; administrator functionality; inventory and reports

The areas map to modules as follows. Each module has exactly one owner, and the owner designs it in Phase 2 and implements it in Phase 3.

Team member	Roll number	Backend modules (Phase 3)	Frontend modules (Phase 3)
Anagha N	PES1UG24CS058	BM-01 Authentication BM-02 User and Profile BM-13 Notification BM-15 Platform and Security	FM-01 Authentication UI FM-08 Profile and Seller Application UI FM-11 Shared UI Infrastructure
Manasvi B	PES1UG24CS261	BM-03 Catalog BM-05 Wishlist BM-08 Review BM-14 Media BM-16 Book Management	FM-02 Catalog UI FM-04 Wishlist UI FM-07 Reviews UI
Meda Sahasra Siri	PES1UG24CS267	BM-04 Cart BM-06 Order BM-07 Payment	FM-03 Cart UI FM-05 Checkout and Payment UI FM-06 Orders UI
Monika	PES1UG24CS274	BM-09 Seller BM-10 Inventory BM-11 Admin BM-12 Reporting	FM-09 Seller Dashboard UI FM-10 Administrator Dashboard UI

How the less obvious modules were placed:

Module	Owner	Reason
BM-13 Notification and the email and payment adapter wiring	Anagha N	Integration of external services and cross-module coordination
BM-05 Wishlist, BM-08 Review, BM-14 Media	Manasvi B	They belong to the catalog experience: book pages, reviews and book images
BM-16 Book Management	Manasvi B	This is the book-management area: seller listings and their search fields
BM-10 Inventory	Monika	Inventory is a seller function (stock and low-stock alerts). Checkout (Meda) calls it for reservations through interface I-09, which both review
FM-08 Profile and Seller Application UI	Anagha N	It is the user-profile screen group; the seller application backend (BM-09) is owned by Monika

9.2 Phase 2 document responsibilities

Owner	Phase 2 design responsibility
Anagha N	Architecture document: overview, diagram, deployment, security architecture, performance and reliability, decisions list. Design document: conventions, ABI, access matrix, authentication and platform modules (BM-01, BM-02, BM-13, BM-15), accounts class diagram, component diagrams, SD-01, SD-02. Validation update: architecture-based validation, security validation, availability test (PT-05). Final consistency audit and document assembly

Owner	Phase 2 design responsibility
Manasvi B	Catalog, search, book management, wishlist, review and media modules (BM-03, BM-05, BM-08, BM-14, BM-16), search design (Section 4.4), catalog class diagram, ER diagram 1, GridFS section, SD-03, SD-09, SD-11, screens S-01, S-02, S-09, S-15, S-19, S-20. Validation update: catalog and search tests, PT-01, PT-02, PT-06
Meda Sahasra Siri	Cart, order and payment modules (BM-04, BM-06, BM-07), reservation algorithms (Section 4.3), purchasing class diagram, service-layer class diagram, ER diagram 2, SD-04 to SD-07, AD-01, screens S-08, S-10 to S-14. Validation update: checkout, payment and order tests, PT-03, PT-04
Monika	Seller, inventory, administration and reporting modules (BM-09, BM-10, BM-11, BM-12), SD-08, SD-10, SD-12, AD-02, AD-03, screens S-17, S-18, S-21 to S-29. Validation update: seller, inventory, admin and report tests, updated traceability matrix

API definitions and endpoint tests follow module ownership: the owner of a module defines and tests its endpoints.

Owner	Modules that serve endpoints	Endpoints defined and tested
Anagha N	BM-01, BM-02, BM-13, BM-15	API-01 to API-14, API-45, API-46, API-61 (17 endpoints)
Manasvi B	BM-03, BM-05, BM-08, BM-14, BM-16	API-15 to API-19, API-24 to API-26, API-33, API-36 to API-39, API-41, API-53 to API-59 (21 endpoints)
Meda Sahasra Siri	BM-04, BM-06, BM-07	API-20 to API-23, API-27 to API-32 (10 endpoints)
Monika	BM-09, BM-10, BM-11, BM-12	API-34, API-35, API-40, API-42 to API-44, API-47 to API-52, API-60 (13 endpoints)

9.3 Review and support responsibilities

Every owner's area is reviewed by one other member, and every area has a second member who helps test it. The assignment is a cycle, so everyone reviews one area and supports testing in another.

Area owner	Primary reviewer	Test and acceptance support
Anagha N	Manasvi B	Monika
Manasvi B	Meda Sahasra Siri	Anagha N
Meda Sahasra Siri	Monika	Manasvi B
Monika	Anagha N	Meda Sahasra Siri

Interfaces that join two owners are reviewed by both owners:

Interface	Between	Reviewed by
I-06 Cart to Inventory, I-09 Order to Inventory	Meda Sahasra Siri and Monika	Both
I-14 Review to Order, I-15 Review to Catalog	Manasvi B and Meda Sahasra Siri	Both
I-18 Fulfilment to Order	Monika and Meda Sahasra Siri	Both
I-10, I-13 callers of Notification	Anagha N with Meda Sahasra Siri and Monika	All three
I-16, I-28 Book Management to Media and Catalog	Manasvi B	Manasvi B and Anagha N

Working rules for Phase 3: each change to an endpoint or interface table is reviewed by the owner of the other side; each member writes the unit tests of the modules they own; the test and acceptance support member runs the use case walkthroughs of the area; and the reviewer checks the traceability matrix rows of the area.

9.4 Cross-cutting Phase 3 tasks

Task	Owner
Repository set-up, continuous integration and deployment	Anagha N
Fake payment gateway and fake mailer for tests	Meda Sahasra Siri
Seed scripts and test fixtures	Manasvi B
End-to-end test suite	Monika
Load-test scripts	Manasvi B
Security scans and dependency checks	Anagha N

10. Traceability: Requirements to Modules and API

The table links every requirement to its use cases, architecture components and design modules. The complete matrix, which adds API endpoints and test cases, is in the Validation Specification update, Section 5.

Req	Use cases	Architecture components	Design modules
FR-01	UC-04, UC-25	AC-01 to AC-05, AC-07	BM-01, BM-13, FM-01
FR-02	UC-05	AC-01 to AC-05	BM-01, FM-01
FR-03	UC-14, UC-25	AC-01 to AC-05, AC-07	BM-01, BM-13, FM-01
FR-04	UC-01	AC-01 to AC-05	BM-03, FM-02
FR-05	UC-02	AC-01 to AC-05	BM-03, FM-02
FR-06	UC-02	AC-01 to AC-05	BM-03, FM-02
FR-07	UC-03	AC-01 to AC-05	BM-03, BM-14, FM-02
FR-08	UC-06	AC-01 to AC-05	BM-04, FM-03
FR-09	UC-06	AC-01 to AC-05	BM-04, BM-10, FM-03
FR-10	UC-07	AC-01 to AC-05	BM-05, FM-04
FR-11	UC-08	AC-01 to AC-05	BM-06, FM-05
FR-12	UC-08	AC-01 to AC-05	BM-04, BM-06, FM-03, FM-05
FR-13	UC-08, UC-09	AC-01 to AC-06	BM-06, BM-07, FM-05
FR-14	UC-08, UC-25	AC-01 to AC-05, AC-07	BM-06, BM-13, FM-05
FR-15	UC-10	AC-01 to AC-05	BM-06, FM-06
FR-16	UC-11	AC-01 to AC-06	BM-06, BM-07, BM-10, FM-06
FR-17	UC-12	AC-01 to AC-05	BM-08, FM-07
FR-18	UC-13	AC-01 to AC-05	BM-02, FM-08
FR-19	UC-15, UC-21	AC-01 to AC-05	BM-02, BM-09, BM-11, FM-08
FR-20	UC-16	AC-01 to AC-05	BM-14, BM-16, FM-09
FR-21	UC-17	AC-01 to AC-05, AC-07	BM-10, BM-13, FM-09
FR-22	UC-18	AC-01 to AC-05	BM-09, FM-09
FR-23	UC-19	AC-01 to AC-05	BM-12, FM-09
FR-24	UC-05, UC-20	AC-01 to AC-05	BM-01, BM-02, BM-11, FM-10
FR-25	UC-21	AC-01 to AC-05	BM-09, BM-11, BM-16, FM-10
FR-26	UC-23	AC-01 to AC-05	BM-03, BM-11, FM-10
FR-27	UC-22	AC-01 to AC-05	BM-08, BM-11, FM-10
FR-28	UC-24	AC-01 to AC-05	BM-11, BM-12, FM-10
FR-29	UC-05 to UC-24 (all role-restricted use cases)	AC-01 to AC-05	BM-01, BM-15, FM-11
FR-30	UC-08	AC-01 to AC-05	BM-06, BM-10
VFR-01	UC-01, UC-02, UC-03	AC-01, AC-02, AC-05	BM-03, BM-15, FM-11
VFR-02	UC-01, UC-02	AC-02 to AC-05	BM-03, BM-15
VFR-03	UC-01 to UC-25 (system-wide)	AC-02 to AC-05	BM-15
VFR-04	UC-04, UC-05, UC-14	AC-03 to AC-05	BM-01
VFR-05	UC-01 to UC-25 (system-wide)	AC-01, AC-02, AC-06, AC-07	BM-15
VFR-06	UC-01 to UC-25 (system-wide)	AC-01 to AC-04	BM-15
VFR-07	UC-05	AC-02 to AC-05	BM-01, BM-15
VFR-08	UC-09	AC-01, AC-03, AC-05, AC-06	BM-07, BM-15
VFR-09	UC-01 to UC-25 (system-wide)	AC-02, AC-08	BM-15
VFR-10	UC-01, UC-02, UC-03, UC-06, UC-08, UC-09	AC-01	FM-01, FM-02, FM-03, FM-05, FM-11
VFR-11	UC-01, UC-02, UC-03, UC-06, UC-08, UC-09	AC-01	FM-01, FM-02, FM-03, FM-05, FM-11
VFR-12	UC-01 to UC-25 (system-wide)	AC-01	FM-11
VFR-13	UC-01 to UC-25 (system-wide)	AC-01 to AC-04	BM-15, FM-11
VFR-14	UC-08, UC-09, UC-11	AC-03 to AC-06	BM-06, BM-07, BM-10

Req	Use cases	Architecture components	Design modules
VFR-15	UC-01 to UC-25 (system-wide)	AC-02	BM-15

11. Assumptions and Open Items

The design decisions and assumptions D-01 to D-15, the Phase 1 ambiguities A-01 to A-14 and the TBD items are listed in the Architecture document, Section 11, and apply to this document unchanged. Items in this document that depend on a TBD are: the payment and email adapter implementations (Section 5.2 and 7.2), the pricing configuration (Section 4.3), and the hosting-dependent parts of the deployment diagram (Section 5.4).